

**REAL-TIME INDIAN SIGN LANGUAGE
RECOGNITION USING EFFICIENT
MULTI-FEATURE ATTENTION MECHANISM**

A PROJECT REPORT

Submitted by

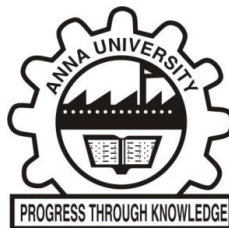
**RAJESH A
KATHIR KALIDASS B
SATHYANARAYANAN P
NANDHA S**

in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



COLLEGE OF ENGINEERING GUINDY

ANNA UNIVERSITY : CHENNAI 600 025

MARCH 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**REAL-TIME INDIAN SIGN LANGUAGE RECOGNITION USING EFFICIENT MULTI-FEATURE ATTENTION MECHANISM**” is the bona fide work of **Rajesh A (2023103046)**, **Kathir Kalidass B (2023103546)**, **Sathyanarayanan P (2023103579)** and **Nandha S (2023103703)** who carried out the project work under my supervision.

SIGNATURE

DR. V. MARY ANITA RAJAM

HEAD OF THE DEPARTMENT

Department of Computer Science
and Engineering,

Anna University,
Chennai 600025.

SIGNATURE

DR. C. VALLIYAMMAI

SUPERVISOR

Professor,
Department of Computer Science
and Engineering,

Anna University,
Chennai 600025.

ACKNOWLEDGEMENT

We express our deep gratitude to our guide, **Dr. C. Valliyammai**, Professor, Department of Computer Science and Engineering, for guiding us through every phase of the project. We appreciate her thoroughness, tolerance, and ability to share her knowledge with us. We would also like to thank her for her kind support and for providing the necessary facilities to carry out this work.

We are extremely grateful to **Dr. V. Mary Anita Rajam**, Professor and Head of the Department, Department of Computer Science and Engineering, Anna University, Chennai-25, for extending the facilities of the Department towards our project and for her unstinting support.

We express our sincere thanks to the esteemed panel of reviewers: **Dr. R. Baskaran**, Professor, Department of Computer Science and Engineering; **Dr. V. Vetrivelvi**, Professor, Department of Computer Science and Engineering; and **Dr. P. Shanthi**, Assistant Professor, Department of Computer Science and Engineering, for their valuable suggestions and critical reviews throughout the course of our project.

We express our thanks to all other teaching and non-teaching staff who helped us in one way or another for the successful completion of the project. We would also like to thank our parents, family, and friends for their indirect contribution to the successful completion of this project.

RAJESH A

KATHIR KALIDASS B

SATHYANARAYANAN P

NANDHA S

ABSTRACT

Communication barriers between hearing-impaired individuals and the general population remain a significant social challenge. Indian Sign Language (ISL) is the primary communication medium for many deaf and hard-of-hearing individuals in India, but the lack of accessible translation tools often restricts interaction with non-signers. To address this problem, this work proposes a real-time Indian Sign Language recognition system based on an Efficient Multi-Feature Attention Mechanism.

The proposed system processes live video streams captured from a camera and converts sign gestures into meaningful text. Video frames are extracted and preprocessed to maintain uniform spatial and temporal representation. Spatial visual cues, such as hand shape, orientation, and appearance, are learned using an EfficientNet model pretrained on ImageNet through transfer learning. In parallel, temporal motion patterns are captured using a CNN–RNN architecture.

To improve recognition performance, an attention mechanism dynamically assigns importance to spatial and temporal features, emphasizing relevant gesture components while suppressing less informative information. The weighted features are fused and passed to a GRU-based sequence modeling network, and a fully connected softmax classifier predicts the corresponding ISL gesture class. Designed for real-time deployment, the framework maintains low computational overhead while delivering reliable recognition accuracy and provides a strong foundation for future work on continuous sign language translation.

திட்டப்பணி சுருக்கம்

கேள்வித்திறன் குறைபாடு உடைய நபர்கள் மற்றும் பொதுமக்கள் இடையேயான தொடர்பு இடைவெளி இன்னும் ஒரு முக்கியமான சமூகச் சவாலாக உள்ளது. இந்தியச் சைகை மொழி, இந்தியாவில் செவிடு மற்றும் கேள்வித்திறன் குறைபாடு உடையோரின் முக்கிய தொடர்பு மொழியாக விளங்கினாலும், அதை உடனடியாக உரை வடிவமாக மாற்றும் எளிய மற்றும் அணுகத்தக்க கருவிகள் குறைவாக இருப்பதால், சைகை மொழியை அறியாதவர்களுடன் தொடர்பு கொள்ளுவதில் சிரமம் ஏற்படுகிறது. இந்தப் பிரச்சினைக்கு தீர்வாக, இவ்வாய்வு பல அம்சங்களை கருத்தில் கொண்டு செயல்படும் கவன ஒதுக்கீட்டு முறையை அடிப்படையாகக் கொண்ட நேரடி இந்தியச் சைகை மொழி அடையாள அமைப்பை முன்வைக்கிறது.

முன்மொழியப்பட்ட அமைப்பு, ஒளிப்பதிவு கருவி மூலம் பெறப்படும் தொடர்ச்சியான கானொளிக் காட்சிகளை செயலாக்கி, சைகை இயக்கங்களை பொருள் கொண்ட உரையாக மாற்றுகிறது. கானொளிப் படங்கள் எடுக்கப்பட்டு, அவை ஒரே மாதிரியான இடவியல் மற்றும் காலவியல் அமைப்பைப் பேணும் வகையில் முன்செயலாக்கம் செய்யப்படுகின்றன. கை வடிவம், திசை, தோற்றம் போன்ற இடவியல் காட்சி அம்சங்கள் முன்பயிற்சி செய்யப்பட்ட திறமையான மாதிரியின் மூலம் கற்றுக்கொள்ளப்படுகின்றன. அதே சமயம், இயக்கங்களின் காலவரிசை வடிவங்கள் இணைந்த நரம்புக் கட்டமைப்பின் மூலம் பதிவுசெய்யப்படுகின்றன.

அடையாளத் துல்லியத்தை மேம்படுத்த, கவன ஒதுக்கீட்டு முறை இடவியல் மற்றும் காலவியல் அம்சங்களுக்கு தன்னிச்சையாக முக்கியத்துவம் வழங்கி, முக்கியமான சைகை கூறுகளை முன்னிறுத்தி, தொடர்பற்ற கூறுகளைத் தணிக்கிறது. இவ்வாறு எடையிடப்பட்ட அம்சங்கள் ஒன்றிணைக்கப்பட்டு, தொடர் சார்ந்த கற்றல் அமைப்பிற்கு அனுப்பப்படுகின்றன. பின்னர், முழுமையாக இணைக்கப்பட்ட வகைப்பாட்டு அமைப்பின் மூலம் பொருத்தமான இந்தியச் சைகை மொழி வகை கணிக்கப்படுகிறது.

நடைமுறை பயன்பாட்டை கருத்தில் கொண்டு வடிவமைக்கப்பட்ட இந்த அமைப்பு, குறைந்த கணக்கீட்டு சுமையுடன் செயல்பட்டு நம்பகமான அடையாளத் துல்லியத்தை வழங்குகிறது. மேலும், தொடர்ச்சியான சைகை மொழி வாக்கியங்களை இயல்பான மொழியாக மாற்றும் எதிர்கால ஆய்வுகளுக்கு இது வலுவான அடித்தளமாக அமைகிறது.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	திட்டப்பணி சுருக்கம்	iv
	LIST OF TABLES	x
	LIST OF FIGURES	xi
	LIST OF ABBREVIATIONS	xii
	LIST OF SYMBOLS	xiii
1	INTRODUCTION	1
1.1	BACKGROUND OF THE STUDY	1
1.2	PROBLEM STATEMENT	2
1.3	OBJECTIVES OF THE PROJECT	3
1.4	SCOPE OF THE WORK	4
1.5	MOTIVATION	5
1.6	OUTLINE OF THE THESIS	6
2	RELATED WORK	7
2.1	OVERVIEW OF EXISTING SYSTEMS	7
2.2	EXISTING METHODS AND TECHNIQUES	8
2.3	STRENGTHS AND LIMITATIONS	11
2.3.1	Advantages	11
2.3.2	Limitations	11
2.4	RESEARCH GAP IDENTIFIED	13

3	PROPOSED METHODOLOGY AND SYSTEM DESIGN	15
3.1	OVERALL SYSTEM ARCHITECTURE	15
3.1.1	Block Diagram	17
3.1.1.1	Video Input and Preprocessing	17
3.1.1.2	RGB Backbone Spatial Features	18
3.1.1.3	Pose-Landmark Stream Features	18
3.1.1.4	Multi-Feature Attention Mechanism	19
3.1.1.5	Feature Fusion	19
3.1.1.6	Sequence Modeling	20
3.1.1.7	Temporal Alignment Using CTC	20
3.1.1.8	Decoding Strategy (Beam and Greedy)	20
3.1.1.9	Language and Grammar Refinement	21
3.1.1.10	Output and Text-to-Speech	21
3.2	MODULES DESCRIPTION	22
3.2.1	Video Acquisition and Preprocessing	22
3.2.1.1	Frame Extraction and Normalization	22
3.2.1.2	Pose and Landmark Extraction	23
3.2.2	Feature Extraction and Fusion	24
3.2.2.1	RGB Feature Extraction Using ResNet Backbone	24
3.2.2.2	Pose Encoding and Gated Fusion	25
3.2.3	Continuous Sign Recognition	26
3.2.3.1	Temporal Modeling	26
3.2.3.2	Sign Prediction Using CTC Decoder	27
3.2.4	Language and Audio Output	29
3.2.4.1	Sentence Correction and Caption Buffer	29
3.2.4.2	Text-to-Speech Output	31
3.3	ALGORITHM	31
3.3.1	Video Stream Capture and Session Initialization	31

3.3.2	Frame Sampling, Cleaning, and Normalization	32
3.3.3	RGB Backbone Feature Extraction	32
3.3.4	Pose Stream Feature Encoding	33
3.3.5	Gated Feature Prioritization and Fusion	33
3.3.6	Multimodal Feature Fusion	33
3.3.7	Sequence Context Modeling	34
3.3.8	Gesture Token Prediction and Validation	34
3.3.9	Language Rendering and Speech Synthesis	34
3.4	MATHEMATICAL MODEL / EQUATIONS	35
3.4.1	Input Standardization	35
3.4.2	Spatial and Pose Feature Encoding	36
3.4.3	Implemented Fusion Models	36
3.4.4	Temporal Modeling and Logit Projection	37
3.4.5	CTC Objective and Decoding	37
3.4.6	Confidence and Output Rendering	38
3.4.7	Evaluation Metrics	38
3.5	WORKFLOW / PROCESS FLOW	39
3.5.1	Operational Modes and Runtime Routing	39
3.5.2	Batch Inference Process Flow	39
3.5.3	Live Streaming Process Flow (Sliding Window)	41
3.5.4	Training-to-Deployment Process Flow	41
3.5.5	Process Integrity and Failure Handling	42
4	IMPLEMENTATION AND RESULTS	43
4.1	IMPLEMENTATION ENVIRONMENT	43
4.1.1	Software and Framework Stack	43
4.1.2	Model Execution Environment	44
4.1.3	Deployment and Runtime Configuration	45
4.1.4	Practical Hardware Profile	45
4.2	DATASET / INPUT DESCRIPTION	46

4.2.1	Dataset Source and Organization	46
4.2.2	Model Input Representation	46
4.2.3	Preprocessing and Standardization	47
4.2.4	Inference Input Modes	48
4.3	TEST CASES	48
4.3.1	Test Scope and Strategy	48
4.3.2	Real-Time Word-Level Test Cases	49
4.4	RESULTS AND DISCUSSION	51
4.4.1	Key Observations	51
4.4.2	Performance Metrics	51
4.4.3	Intermediate Outputs	54
4.4.4	Result Analysis	55
5	CONCLUSION AND FUTURE WORK	57
5.1	SUMMARY OF THE WORK	57
5.2	ACHIEVEMENTS	58
5.3	LIMITATIONS	59
5.4	FUTURE WORK	60
	APPENDIX	62
1	ADDITIONAL MATERIAL	62
1.1	KEY ALGORITHMS	62
1.2	REPRODUCIBILITY CHECKLIST	63
1.3	CONFIGURATION SNAPSHOT	63
	REFERENCES	64

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Representative SLR systems: methods and limits	10
2.2	Major SLR families: advantages and limits	12
4.1	Real-time test cases with matched frames	50
4.2	Performance comparison: references vs this work	52
A 1.1	Representative configuration snapshot	63

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Architecture diagram of proposed model.	17
3.2	Video Acquisition and Preprocessing Module.	24
3.3	Feature Extraction and Fusion Module.	26
3.4	Continuous Sign Recognition Module.	29
3.5	Language and Audio Output Module.	30
4.1	Dataset and input representation snapshot.	48
4.2	Test case setup and evaluation snapshot.	50
4.3	Heatmap view of key performance metrics	53
4.4	Accuracy and F1-score comparison across	53
4.5	Latency versus WER trade-off comparison	54
4.6	Intermediate output snapshot 1.	55
4.7	Intermediate output snapshot 2.	55

LIST OF ABBREVIATIONS

ISL	-	Indian Sign Language
SLR	-	Sign Language Recognition
CSLR	-	Continuous Sign Language Recognition
CNN	-	Convolutional Neural Network
RNN	-	Recurrent Neural Network
LSTM	-	Long Short-Term Memory
BiLSTM	-	Bidirectional Long Short-Term Memory
GRU	-	Gated Recurrent Unit
CTC	-	Connectionist Temporal Classification
TTS	-	Text-to-Speech
RGB	-	Red-Green-Blue color representation
FPS	-	Frames per second
WER	-	Word Error Rate
CER	-	Character Error Rate
BLEU	-	Bilingual Evaluation Understudy score
ROUGE	-	Recall-Oriented Understudy for Gisting Evaluation

LIST OF SYMBOLS

X_t	-	RGB frame at time step t
P_t	-	Pose-landmark feature vector at time step t
T	-	Number of frames in an input sequence
F_t^{rgb}	-	Spatial feature vector extracted from X_t
F_t^{pose}	-	Pose feature vector extracted from P_t
α_t	-	Attention weight for RGB features at time step t
β_t	-	Attention weight for pose features at time step t
F_t^{fuse}	-	Attention-fused multimodal feature at time step t
H_t	-	Temporal hidden representation after sequence encoding
Y	-	Ground-truth output label sequence
$\hat{Y}$	-	Predicted output label sequence
L_{CTC}	-	Connectionist Temporal Classification loss
TP	-	True Positives
TN	-	True Negatives
FP	-	False Positives
FN	-	False Negatives
Acc	-	Accuracy
Prec	-	Precision
Rec	-	Recall
F_1	-	F1-score
S, D, I	-	Substitutions, Deletions, Insertions (word-level)
N_w	-	Total number of words in reference sequence
S_c, D_c, I_c	-	Substitutions, Deletions, Insertions (character-level)
N_c	-	Total number of characters in reference sequence

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND OF THE STUDY

Communication is a fundamental requirement for social participation, education, healthcare access, and economic opportunity. For members of the deaf and hard-of-hearing community in India, Indian Sign Language (ISL) is a natural and expressive medium that encodes meaning through coordinated hand shape, hand orientation, movement trajectory, body posture, and facial expression. In practice, however, smooth communication between ISL users and non-signers is still limited by the availability of trained interpreters and by the absence of reliable real-time assistive tools in many public contexts.

The research community has shown that deep learning methods can significantly improve visual recognition tasks, including gesture and sign understanding. Modern computer vision models can learn robust spatial representations from raw RGB frames, while sequence models can capture temporal evolution across consecutive frames. These developments are strongly aligned with sign language understanding, where meaning depends not only on static hand configuration but also on motion continuity and contextual timing [5, 9, 13].

In the Indian deployment context, the challenge is not only recognition quality but also operational stability. A practical system must sustain consistent behavior under variable lighting, camera motion, and signer-specific style differences. It must also remain responsive enough for conversational use, where delay directly affects communication flow and user trust.

Therefore, the background of this study is framed around a dual objective: achieving robust visual-temporal understanding and ensuring real-time usability. This framing motivates an architecture that combines efficient feature extraction, temporal modeling, and selective attention so the system can remain accurate without becoming too heavy for practical inference.

Despite this progress, ISL-focused systems still face practical and technical constraints. Gesture classes can be visually similar, signer-specific variations can be large, and unconstrained environments introduce noise from lighting, background clutter, and camera perspective. Therefore, there is a clear need for an ISL recognition framework that is both accurate and computationally efficient, and that can operate with real-time responsiveness for deployment-oriented scenarios.

1.2 PROBLEM STATEMENT

Although existing sign language recognition approaches report encouraging results on benchmark datasets, many systems remain difficult to deploy in real-time communication settings. A common limitation is the imbalance between spatial and temporal modeling: models that focus heavily on frame-level appearance may miss motion continuity, whereas sequence-heavy models may lose fine-grained hand configuration cues. This trade-off directly affects recognition quality for signs that differ by subtle movement or short-duration transitions.

Another challenge is computational efficiency. High-capacity architectures may improve offline accuracy, but they often increase latency and memory usage, reducing practical usability for continuous camera input. For assistive communication, response delay must be low enough to preserve conversational flow. In addition, ISL-specific data diversity is still evolving, and variability

across signer style, speed, and environmental conditions can reduce generalization when models are trained without robust feature selection and fusion strategies.

To make the problem operationally measurable, this thesis focuses on three guiding questions: how to preserve fine-grained spatial detail while modeling temporal dynamics, how to maintain low-latency inference under realistic video conditions, and how to improve robustness against signer and environment variability without excessive model complexity. These questions define the practical boundary between purely academic performance and deployment-oriented utility.

The core problem addressed in this thesis is therefore the design of a real-time ISL recognition system that can jointly model spatial detail and temporal dynamics while remaining computationally efficient. The system must maintain stable performance under realistic input variability and produce output that can directly support communication between ISL users and non-signers.

1.3 OBJECTIVES OF THE PROJECT

The proposed work is aimed at developing an assistive ISL recognition pipeline that is academically rigorous and practically deployable. In line with this goal, the project objectives are defined to balance algorithmic performance, model efficiency, and usability of generated output.

The specific objectives are as follows:

- To design and implement a real-time ISL recognition architecture using RGB video input.
- To extract discriminative spatial features using EfficientNet-based visual encoding.

- To capture gesture dynamics through temporal modeling over frame sequences.
- To integrate a multi-feature attention mechanism that emphasizes informative visual-temporal cues.
- To generate usable communication output by converting recognized signs into readable text and speech.

Along with these functional goals, the project also targets predictable runtime behavior and modular implementation. This ensures that each stage, from preprocessing to output generation, can be profiled, improved, and reused for future extensions such as continuous sentence-level recognition and multilingual assistive workflows.

Collectively, these objectives define a complete path from visual acquisition to assistive output, with emphasis on both recognition reliability and real-time behavior.

1.4 SCOPE OF THE WORK

This thesis focuses on the recognition of ISL gestures from RGB video streams using a deep learning pipeline that combines spatial feature extraction, temporal modeling, and attention-based feature refinement. The implementation scope includes data preprocessing, model design, training and inference workflow, and output generation as text and speech. A key requirement within this scope is real-time oriented performance, so architectural choices are made with computational efficiency in mind rather than only maximizing offline benchmark accuracy.

The present work is intentionally bounded to maintain technical clarity and measurable outcomes. The system is developed primarily for isolated

gesture recognition and structured evaluation under controlled experimental settings. While the framework can inform future extensions, full continuous sentence-level translation, multilingual sign integration, and edge deployment on low-power embedded hardware are treated as beyond the immediate scope of this thesis and are discussed as forward directions.

Within this scope, certain assumptions are maintained for experimental consistency. The input stream is assumed to provide sufficiently visible upper-body gesture regions, and training data is assumed to represent the target sign classes with reasonable class balance. These assumptions help isolate modeling and inference behavior, while still preserving practical relevance for future real-time deployment refinement.

1.5 MOTIVATION

The primary motivation for this work is the need to reduce communication barriers faced by ISL users in everyday interactions. In academic institutions, hospitals, offices, and public service centers, communication often depends on interpreter availability, which may not always be practical. A robust automated recognition system can act as an assistive bridge, enabling faster and more inclusive interaction between signers and non-signers.

From a technical perspective, ISL recognition is a meaningful and challenging problem at the intersection of computer vision, sequence modeling, and human-centered AI. It requires the model to understand both fine-grained visual structure and temporal progression under real-world variability. Addressing this challenge contributes not only to assistive communication technology but also to broader research on efficient multimodal understanding systems.

The motivation is also driven by usability expectations from end users. A system that is highly accurate in offline tests but unstable in live settings cannot

serve assistive communication effectively. Hence, the present work prioritizes robustness, interpretability of outputs, and practical response time together, instead of treating these factors as secondary goals.

This thesis is therefore motivated by a combined social and engineering objective: to build a system that is both practically useful and technically sound, with a clear pathway toward future real-world deployment.

1.6 OUTLINE OF THE THESIS

This thesis is organized into five chapters, each addressing a specific stage of the study. Chapter 1 introduces the communication context, problem definition, objectives, scope, and motivation for the proposed work. Chapter 2 reviews existing literature and related methods in sign language recognition, followed by a critical analysis of current limitations and research gaps. Chapter 3 presents the proposed methodology, including system architecture, module-level design, algorithmic flow, and supporting mathematical formulation.

Chapter 4 describes implementation details, dataset and input handling, experimental setup, and performance-oriented discussion of results. Finally, Chapter 5 summarizes the major contributions of the thesis, discusses practical limitations, and outlines directions for future enhancement. This progression is intended to provide a coherent transition from problem framing to methodological design, empirical validation, and concluding insights.

This structure also supports focused reading. Readers interested in model design can move directly from Chapters 1 and 2 to Chapter 3, while readers focused on practical outcomes can follow Chapters 3 and 4 together. The chapter flow is intentionally aligned so that each methodological decision is connected to measurable results and deployment implications.

CHAPTER 2

RELATED WORK

2.1 OVERVIEW OF EXISTING SYSTEMS

Sign language research has evolved from handcrafted-feature pipelines to large-scale deep learning systems. Early recognition systems were typically constrained to small vocabularies, controlled capture settings, or handcrafted descriptors, and therefore struggled to generalize to real communication scenarios. The deep learning transition, anchored by representation learning advances [5], enabled end-to-end feature discovery from visual data and significantly improved recognition quality.

Modern systems are now commonly organized along two dimensions: task complexity and modality design. In terms of task complexity, the field progressed from isolated sign recognition to continuous sign language recognition (CSLR), and then to full sign language translation (SLT), where sign sequences are mapped to spoken-language sentences with different grammar and word order [9, 14]. In terms of modality design, approaches range from RGB-only pipelines to multimodal pipelines that combine RGB with pose, depth, or linguistic priors for better robustness.

Dataset growth has been central to this progress. Large public benchmarks such as MS-ASL and WLASL improved word-level recognition research [13, 15], while RWTH-PHOENIX-Weather and How2Sign supported continuous recognition and translation benchmarking [9, 16]. More recent resources, such as Sem-Lex and iSign, further expanded coverage through phonological annotations and large ISL-aligned text-video pairs [2, 10]. This dataset expansion

has shifted the field from proof-of-concept models toward scalable and reproducible evaluation.

From a systems perspective, this evolution also reflects a shift from model-centric design to pipeline-centric design. Earlier studies primarily reported top-line accuracy, whereas recent work increasingly reports end-to-end behavior including preprocessing stability, temporal consistency, and inference-time feasibility. As a result, current literature is better viewed as an ecosystem of interconnected components (data, feature extraction, temporal modeling, alignment, and decoding) rather than isolated architecture improvements.

Another notable trend is the growing emphasis on reproducibility and standardized reporting. As datasets become larger and methods become more complex, fair comparison now requires explicit discussion of training splits, preprocessing assumptions, and inference settings. This methodological transparency is increasingly treated as a core requirement for progress in CSLR and ISL-specific research.

At the same time, recent surveys report that high offline performance does not automatically guarantee deployment readiness. Generalization across signers, domains, and devices remains challenging, and real-time constraints continue to expose gaps in latency, robustness, and practical usability [11, 12]. Therefore, existing systems should be understood as strong but still transitional steps toward reliable real-time assistive communication.

2.2 EXISTING METHODS AND TECHNIQUES

Current sign language systems are predominantly built as spatiotemporal learning pipelines. The spatial stage encodes hand shape, orientation, and local appearance cues, often through transfer-learning backbones or 2D/3D convolutional models. Recent work has shown that strong visual backbones such

as EfficientNet can improve representation quality when combined with task-specific contextual refinement [1]. The temporal stage then models sequential dependencies using recurrent architectures (LSTM/GRU), temporal convolutions, or transformer encoders, depending on sequence length, latency constraints, and training scale [8, 14].

A second major direction is sequence alignment and decoding. Since frame-level sign boundaries are hard to annotate at scale, many CSLR systems use CTC-based alignment for weakly aligned supervision. This strategy has become practical for real-time and continuous settings because it supports variable-length input and output sequences. The SignFlow framework, for example, combines pretrained feature extraction, transformer-based temporal learning, and CTC fine-tuning, while introducing device-aware down-sampling and detection-rate analysis for real-time operation in ISL scenarios [4].

A third direction is multimodal and attention-guided fusion. RGB captures rich appearance information, whereas pose and landmark streams offer geometric motion cues that are often less sensitive to background variation. Attention mechanisms are increasingly used to weight these feature streams adaptively, improving robustness for subtle gestures and speed variation [1, 4]. Gloss-aware and gloss-free translation models further extend this line by integrating local semantic boundary modeling and sentence-level language priors [17, 18].

In practical deployments, these method families are rarely used in isolation. Competitive systems often combine transfer-learned visual backbones, lightweight temporal encoders, and alignment-aware decoders in a staged training strategy. This combination allows models to retain discriminative spatial features while still controlling sequence-level drift and runtime overhead, which is especially important for continuous ISL usage in assistive settings.

Finally, dataset-centric methods and pretraining strategies now play a critical role in model quality. Large-scale supervised resources such as iSign and

Sem-Lex provide stronger linguistic and phonological coverage [2, 10], while broad multimodal datasets such as How2Sign improve generalizable sequence modeling [16]. Recent large-scale unification efforts suggest that cross-dataset pretraining and shared representations can further improve transferability across sign languages and downstream tasks [19]. Overall, the field has shifted from single-model optimization to joint optimization of data curation, architecture design, and alignment-aware training.

Table 2.1 summarizes representative systems reviewed in this chapter using a uniform comparison format.

Paper / System	Method Used	Advantage	Limitation
Yenisari and Yavuz (2025) [1]	EfficientNet + CNN/RNN + attention (isolated SLR)	Strong contextual feature learning and high benchmark accuracy	RGB-centric setup; higher run-time cost for large models
Geetha et al. (2025) SignFlow [4]	CNN + Transformer + CTC with RGB + pose (continuous ISL)	Real-time-oriented design with improved WER and domain-specific pretraining	Requires careful pretraining and runtime tuning
Camgoz et al. (2020) [14]	Joint transformer-based end-to-end CSLR/SLT	Unified recognition and translation within one sequence model	Needs large aligned corpora; weaker portability in low-resource settings
Joshi et al. (2024) iSign [2]	Large-scale ISL benchmark with multi-task baselines	Major improvement in ISL data scale and evaluation standardization	Primarily benchmark-focused, not an end-to-end deployment system
Kezar et al. (2023) Sem-Lex [10]	SL-GCN with phonological auxiliary targets	Better few-shot recognition and improved linguistic interpretability	Isolated ASL focus; limited direct coverage for continuous translation

Table 2.1: Representative SLR systems: methods and limits

The comparison indicates a consistent pattern: methods that balance

representation quality, temporal alignment, and computational efficiency perform better for practical ISL usage than methods that optimize only one dimension. This reinforces the need for integrated architecture design and motivates the methodology proposed in this thesis.

2.3 STRENGTHS AND LIMITATIONS

2.3.1 Advantages

Recent approaches demonstrate several clear advantages. Deep visual encoders and temporal models can learn rich gesture representations directly from data, significantly improving recognition quality over handcrafted pipelines. Attention-based fusion further improves reliability by focusing computation on informative spatial and temporal regions, which is valuable for signs with subtle finger or motion distinctions [1, 4]. Large curated datasets and multilingual benchmark expansion also improved comparability and reproducibility in model evaluation [2, 10, 13, 15]. In addition, transformer-based translation frameworks have strengthened sequence-level language modeling and enabled better sentence-level generation than earlier isolated-recognition-only systems [14, 17].

2.3.2 Limitations

Despite these strengths, limitations remain substantial. First, many high-performing models are computationally expensive, making them difficult to deploy in low-latency, low-resource settings. Second, data requirements are still high, and annotation bottlenecks remain severe for continuous sign streams and low-resource sign languages. Third, cross-dataset and cross-signer generalization is often weaker than reported in single-benchmark evaluations, which limits

practical transferability [11, 12]. Fourth, a gap persists between offline benchmark success and real-time assistive usability, particularly under unconstrained camera conditions, variable frame rates, and signer diversity [3, 4]. Therefore, existing systems are technically strong but not yet sufficient as fully reliable real-world communication solutions.

An important practical implication is that deployment quality is determined by trade-offs rather than any single metric. A model with slightly lower benchmark accuracy may still be more useful in real settings if it offers stable frame-rate processing, predictable latency, and reduced sensitivity to camera and signer variation. Hence, robust ISL system design should prioritize balanced performance profiles instead of maximizing isolated leaderboard scores.

Table 2.2 compares major approach families in terms of practical strengths and trade-offs.

Approach Family	Key Advantage	Key Limitation	Deployment Suitability
RGB-only deep models	Simple capture setup; strong appearance learning without extra sensors	Sensitive to background/illumination noise and subtle motion ambiguity	Medium
RGB + pose multimodal models	Better robustness by combining appearance and geometric motion cues	Fusion and synchronization overhead during training/inference	High (with optimization)
Transformer-based sequence models	Strong long-range temporal/context modeling for CSLR/SLT tasks	Higher compute and memory demand; data-hungry behavior	Medium to High
CTC-aligned continuous models	Supports variable-length sequences without strict frame-level labels	May lose fine boundary detail in ambiguous transitions	High
Large-scale pretraining and benchmark-driven methods	Improves transferability and reproducibility across tasks	Strong dependence on domain match and annotation quality	Medium to High

Table 2.2: Major SLR families: advantages and limits

Key observations:

- Real-time ISL systems benefit most from multimodal fusion combined with alignment-aware decoding.

- Benchmark accuracy alone is insufficient; latency and robustness must be reported together.
- Data quality and domain fit remain as important as model architecture choice.

Taken together, these findings suggest that deployment-oriented ISL systems should be evaluated with multi-objective criteria. A robust system is one that maintains acceptable recognition quality while preserving low latency, stable behavior across signers, and transparent performance under variable capture conditions.

2.4 RESEARCH GAP IDENTIFIED

The reviewed literature indicates that the central gap is no longer only model accuracy, but the integration of accuracy, robustness, and real-time deployability in ISL-specific settings. Existing methods provide strong isolated-task results, yet many are still optimized for offline benchmarks rather than continuous assistive interaction. This is particularly critical in Indian contexts, where practical deployment must handle signer variation, unconstrained backgrounds, and device-level frame-rate differences while maintaining low latency.

A second gap is multimodal alignment quality. Although recent works confirm that combining RGB and pose improves recognition robustness, many systems still lack stable fusion mechanisms that remain effective under noisy capture conditions and cross-domain shifts [1, 4]. In continuous recognition and translation, temporal alignment remains a bottleneck, especially when gloss supervision is limited or unavailable [14, 17, 18].

A third gap concerns ISL-focused resources and evaluation depth. The emergence of iSign and new ISL studies is a major step forward, but compared with mature ASL/DGS ecosystems, ISL still needs broader benchmark

standardization, stronger real-time evaluation protocols, and more systematic reporting of deployment metrics beyond offline accuracy [2, 11, 12].

Additionally, many prior studies evaluate components independently (feature extraction, sequence modeling, or decoding) but do not provide integrated analysis of failure propagation across the full pipeline. In real-time operation, small errors introduced at early stages such as frame sampling, landmark estimation, or temporal segmentation can accumulate and significantly affect final recognition quality. This indicates a need for end-to-end evaluation frameworks that explicitly connect component-level behavior with sequence-level outcomes.

Therefore, the research gap addressed in this thesis is the need for an ISL-oriented, dual-stream (RGB + pose), attention-guided, and computationally practical pipeline that is evaluated not only for recognition quality but also for real-time applicability. This gap definition directly motivates the proposed methodology in Chapter 3.

In summary, the gap is not the absence of promising components, but the absence of a consistently integrated real-time ISL framework that links component-level design with deployment-level constraints. This positioning provides the foundation for the architecture and evaluation strategy introduced in the next chapter.

CHAPTER 3

PROPOSED METHODOLOGY AND SYSTEM DESIGN

3.1 OVERALL SYSTEM ARCHITECTURE

The overall architecture of the proposed Indian Sign Language recognition system is designed to process visual gesture input and convert it into meaningful textual and audio output. The system operates through a multi-stage deep learning pipeline that extracts both spatial and temporal information from the video sequence.

The complete workflow of the proposed system is illustrated in Fig. 3.1, which presents the architecture diagram of the model and the interaction between different processing modules. As shown in Fig. 3.1, the system begins with the acquisition of video input captured using a standard RGB camera. The input video is decomposed into individual frames through preprocessing. These frames are resized and normalized to maintain consistent spatial dimensions and improve numerical stability during model training and inference.

The processed frames are passed to a dual-stream feature extraction stage where the system learns complementary gesture evidence. In the implemented modular backend, the RGB stream uses a ResNet-based encoder (ResNet18 by default, with configurable ResNet34/50 variants), while the pose stream encodes aligned landmark vectors through an MLP-based pose encoder. In the checkpoint runtime path, inference can also operate in a pose-first configuration for low-latency deployment. Together, these streams capture appearance cues and geometric motion cues before fusion.

For mathematical consistency across the thesis, the RGB frame at time step t is denoted by X_t and the aligned pose-landmark vector is denoted by P_t . The corresponding extracted feature vectors are represented by F_t^{rgb} and F_t^{pose} , respectively.

After feature extraction, the system applies gated multimodal fusion. The implemented fusion learns modality-specific gates for RGB and pose features and combines them into a unified spatiotemporal representation, followed by normalization for stable downstream temporal modeling.

The fused feature sequence is then processed by a sequence modeling component that captures contextual relationships between frames. The modular path uses a BiLSTM temporal model by default, while the deployed checkpoint path uses a causal temporal-convolution plus self-attention variant; Transformer-style encoding remains a configurable alternative. A Connectionist Temporal Classification (CTC) layer aligns predicted labels with input sequences without requiring precise frame-level annotations. Finally, the recognized gesture sequence is converted into readable text and optionally transformed into speech output using a text-to-speech module.

Thus, the architecture integrates multiple deep learning components to effectively capture gesture appearance, motion dynamics, and contextual relationships, enabling accurate real-time recognition of Indian Sign Language gestures. In Fig. 3.1, green blocks represent established/core CSLR pipeline operations, blue blocks represent advanced modeling enhancements emphasized in this work, magenta blocks indicate project-specific integration and correction modules, and white blocks denote control and interface transitions between stages.

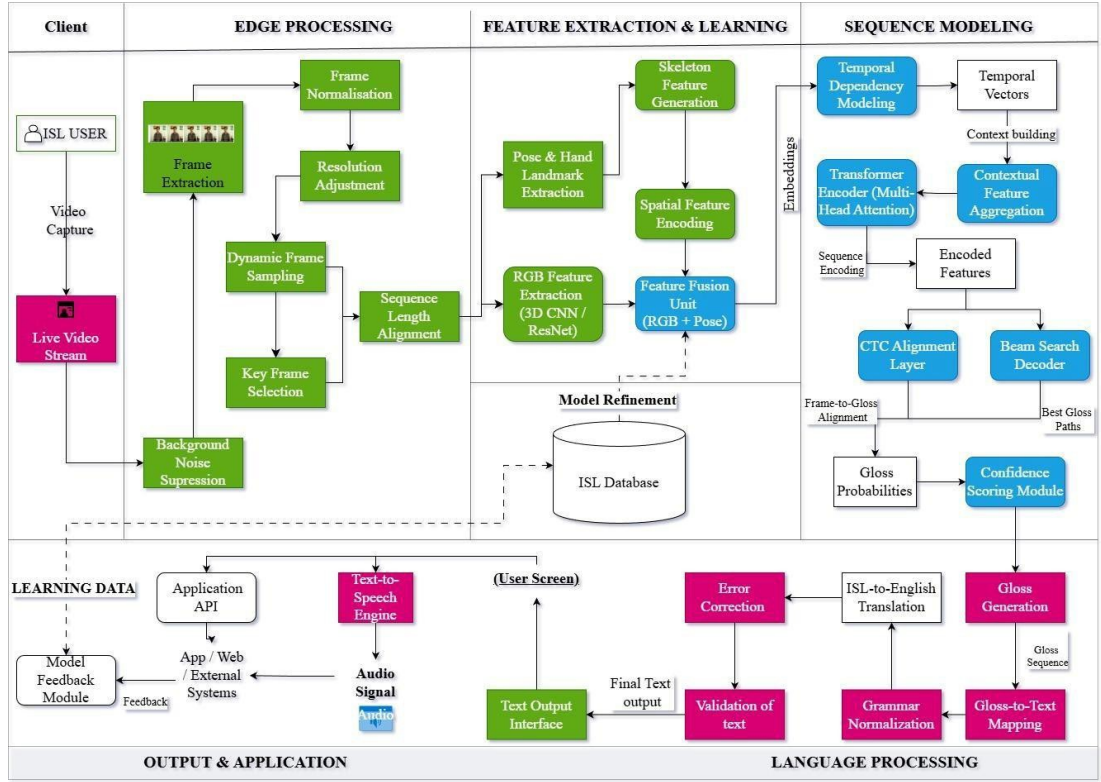


Figure 3.1: Architecture diagram of proposed model.

3.1.1 Block Diagram

3.1.1.1 Video Input and Preprocessing

The system begins with RGB video capture from a live camera or recorded input source and treats this stream as a time-indexed sequence rather than independent images. Because raw capture is noisy and device-dependent, preprocessing acts as a stabilization layer that enforces consistent quality before feature learning starts. In this stage, the stream is decomposed into frames, sampled at a controlled temporal rate, resized to model-compatible dimensions, and normalized to a stable intensity distribution. In addition, frame quality control and sequence standardization are performed so that each sample satisfies minimum temporal completeness. This design is essential for deployment because

recognition quality is highly sensitive to early-stage timing drift, blur accumulation, and irregular sequence lengths. Therefore, preprocessing is not only a data cleaning step but also a reliability control stage that directly impacts downstream model confidence and inference latency.

3.1.1.2 RGB Backbone Spatial Features

The spatial stream uses an RGB backbone encoder to convert each normalized frame into compact high-information embeddings that preserve gesture-defining appearance cues. In the implemented backend, this role is handled by the RGBStream module with ImageNet-pretrained ResNet18 as the default backbone, while ResNet34/50 remain configurable options. The stream captures static and local visual attributes such as hand contour, palm orientation, finger spread, and signer-centric appearance context. This stage provides the primary appearance evidence for recognition and produces feature vectors that are dimension-aligned for multimodal fusion.

3.1.1.3 Pose-Landmark Stream Features

The second stream encodes pose-landmark tensors aligned with each RGB frame. In the modular path, landmark extraction and pose encoding produce geometric features that emphasize joint trajectories, hand articulation, and relative motion patterns. These pose features remain informative even when appearance cues degrade under illumination change, background clutter, or signer variability. Because RGB and pose streams remain time-synchronized, the system can perform frame-wise multimodal fusion with consistent temporal alignment.

3.1.1.4 Multi-Feature Attention Mechanism

The attention module performs adaptive feature prioritization across RGB and pose representations and acts as a selective filter for informative evidence. Instead of treating all frames and channels equally, the model learns relevance scores that elevate discriminative cues and suppress low-value activations caused by background clutter, minor occlusion, or capture instability. This mechanism is central to robust ISL recognition because informative segments are not uniformly distributed across a sequence. By concentrating model capacity on semantically dense regions, the attention stage improves signal-to-noise ratio before fusion and reduces error propagation to decoding layers. In practical terms, attention contributes to better confidence consistency and lower sensitivity to signer-specific style variations.

3.1.1.5 Feature Fusion

After weighting, the RGB and pose representations are fused into a single multimodal embedding that preserves complementary strengths from both streams. Fusion is designed to retain fine-grained appearance details from the RGB branch while simultaneously preserving geometric motion continuity from the pose branch. This unified representation is necessary for complex signs where either appearance-only or pose-only evidence is insufficient for reliable classification. The fusion stage also improves operational robustness by reducing dependence on a single modality under noisy conditions. In the proposed architecture, this block becomes the key transition point from low-level feature encoding to high-level sequence reasoning and is directly linked to recognition quality in continuous input settings.

3.1.1.6 Sequence Modeling

The fused sequence is modeled with a temporal encoder that captures short- and long-range dependencies across time. The modular backend uses BiLSTM by default for stable bidirectional context aggregation, while the deployed iSign checkpoint path uses a causal temporal-convolution block with self-attention for low-latency online behavior. Transformer encoding remains an optional configuration for long-context experimentation. This stage converts fused multimodal features into context-aware embeddings that are aligned for CTC decoding.

3.1.1.7 Temporal Alignment Using CTC

Connectionist Temporal Classification (CTC) provides sequence alignment between input frame streams and output token streams without requiring strict frame-wise annotations. This is critical for ISL datasets where exact gesture boundary labeling is expensive and often inconsistent across annotators. CTC allows the model to learn many-to-one temporal mapping with blank handling and repetition collapse, making it naturally suited for continuous recognition. By removing the need for exact frame-level supervision, CTC improves scalability and simplifies training data preparation. In the overall design, this alignment layer bridges context-rich sequence embeddings and stable gesture token output, thereby enabling practical training and inference under real-world temporal variability.

3.1.1.8 Decoding Strategy (Beam and Greedy)

During inference, decoding strategy is selected according to operating mode. Full-video inference uses beam search to evaluate competing token paths and improve sequence stability, while frame-sequence and live-stream modes

typically use greedy decoding for lower latency. This mixed strategy balances accuracy and responsiveness under real-time constraints. Beam width and confidence thresholds are adjustable so the decoder can be tuned for deployment priorities before language-level correction.

3.1.1.9 Language and Grammar Refinement

The recognized gesture token sequence is mapped into text and then passed through a grammar refinement stage to improve readability, syntactic quality, and communication clarity. Since gloss ordering often differs from standard written language structure, direct token-to-text mapping may produce incomplete or structurally awkward output. The refinement stage performs normalization of word order, removal of repetitive artifacts, and sentence-level correction to produce user-facing text that is easier for non-signers to understand. This stage is essential for practical assistive use because recognition quality alone does not guarantee communicative quality. In effect, language refinement transforms technical recognition output into socially usable content.

3.1.1.10 Output and Text-to-Speech

The final stage renders refined text to the interface and, when enabled, synthesizes speech through a text-to-speech subsystem. Caption rendering supports immediate visual feedback and interaction continuity, while audio output enables communication with listeners who depend on spoken-language output. This dual-mode design improves accessibility and broadens practical usability in educational, service, and public-facing scenarios. The output subsystem also acts as an endpoint for confidence-aware validation, ensuring that unstable predictions can be corrected before presentation. Consequently, the architecture closes the loop from visual capture to multimodal human-consumable communication output.

3.2 MODULES DESCRIPTION

3.2.1 Video Acquisition and Preprocessing

3.2.1.1 Frame Extraction and Normalization

This module is responsible for converting raw video input into a structured and standardized format suitable for deep learning models. The system accepts an RGB video stream captured using a standard camera. Since deep learning models cannot directly operate on continuous video streams, the video is first decomposed into individual frames.

Frame extraction is performed using uniform sampling, where a fixed number of frames are selected from each video sequence. Uniform sampling preserves temporal information while avoiding redundant frames that do not contribute meaningful information. This step is especially important for real-time processing because it reduces computational overhead without losing critical motion cues.

Once extracted, each frame is resized to a predefined spatial resolution compatible with convolutional backbones and normalized to a stable intensity range. In the implemented inference path, sampling is normalized to a target frame rate (25 FPS) before feature extraction. For training and batch inference, sequences are standardized to configured clip lengths; for live mode, the same preprocessing is applied but frames are accumulated in rolling windows rather than fixed full clips.

From a deployment perspective, this module also acts as a reliability gate for the full pipeline. Input stabilization prevents early-stage defects from cascading into later feature and decoding blocks. In practical terms, maintaining consistent frame quality and sequence length improves both convergence during training and confidence stability during inference. This module therefore

combines data engineering and model-readiness objectives: it reduces runtime variability while preserving the temporal structure needed for continuous sign interpretation.

Output: A fixed-length sequence of normalized RGB frames, denoted by $\{X_t\}_{t=1}^T$.

3.2.1.2 Pose and Landmark Extraction

This sub-module extracts structured motion and positional information from video frames. In the modular backend, MediaPipe-based extraction provides body and hand landmarks (75 keypoints in the default path), which are converted into compact pose vectors aligned with each frame. These landmarks encode geometric relationships such as joint positions, hand orientation, and finger movement patterns.

Pose and landmark information provides complementary motion cues that are invariant to background noise and illumination changes. By representing gestures in terms of relative positions and movements, the system improves robustness to environmental variations. The extracted landmark features are temporally aligned with the corresponding RGB frames to preserve synchronization between visual appearance and motion information.

This structured representation is especially useful when appearance cues are weak, for example under low contrast clothing or cluttered backgrounds. Landmark trajectories carry geometric motion information that remains informative even when pixel-level texture is ambiguous. As a result, pose and landmark extraction improves model generalization across signer styles, camera positions, and recording conditions. The alignment between X_t and P_t is preserved throughout the pipeline so that multimodal fusion remains temporally consistent.

Output: A sequence of pose-landmark feature vectors aligned with video frames, denoted by $\{P_t\}_{t=1}^T$.

The end-to-end preprocessing pipeline is shown in Fig. 3.2, where green blocks denote the established preprocessing path, blue highlights denote enhanced structured extraction components, magenta denotes project-specific re-capture or corrective control, and white blocks denote decision and interface nodes.

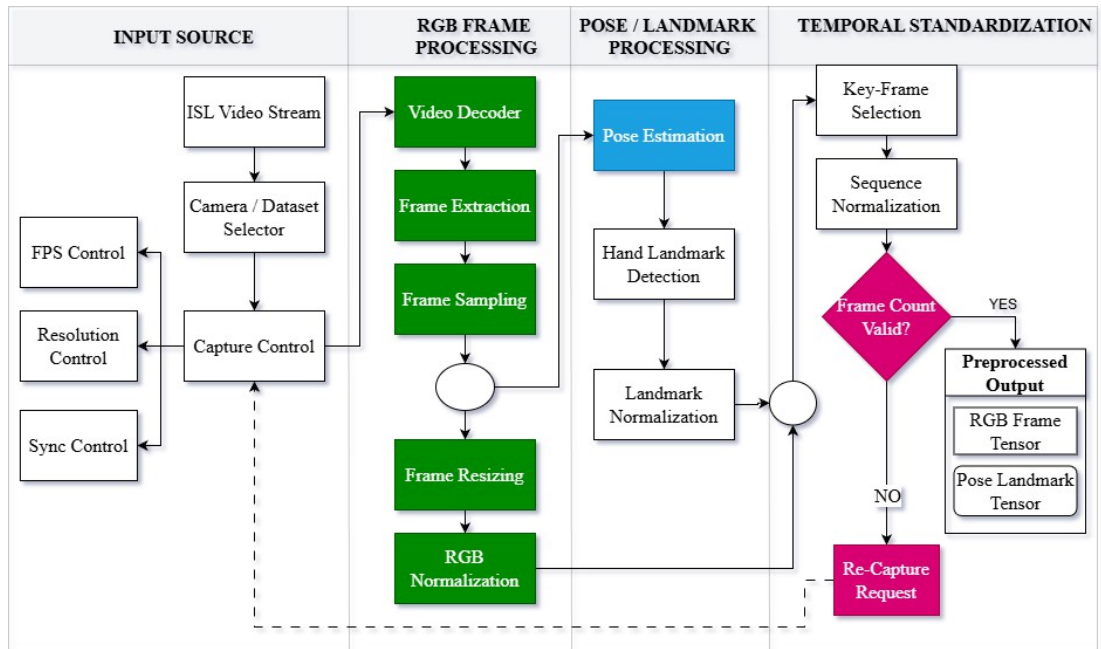


Figure 3.2: Video Acquisition and Preprocessing Module.

3.2.2 Feature Extraction and Fusion

3.2.2.1 RGB Feature Extraction Using ResNet Backbone

This module extracts spatial appearance features from RGB frames using the RGBStream encoder. The default implementation uses ImageNet-pretrained ResNet18, with configurable ResNet34/50 alternatives for different compute-accuracy budgets. Each normalized frame is passed through the backbone to

produce compact feature vectors that encode hand shape, orientation, and other discriminative appearance cues.

The RGB stream is designed for practical deployment: it preserves fine-grained visual information while keeping inference cost manageable. These frame-wise descriptors provide the appearance branch of multimodal recognition and are dimension-aligned with pose features for fusion.

Output: Frame-wise RGB feature vectors, denoted by $\{F_t^{\text{rgb}}\}_{t=1}^T$.

3.2.2.2 Pose Encoding and Gated Fusion

This sub-module processes landmark vectors through a pose encoder and then fuses pose and RGB features with learnable gates. In the modular backend, pose inputs are projected by an MLP encoder (default input formed from aligned landmark coordinates), producing frame-wise pose embeddings with the same feature dimensionality as the RGB stream.

Fusion is implemented with modality-specific sigmoid gates and normalization, so RGB and pose contributions are reweighted adaptively at each time step. This is more robust than fixed-weight fusion when signer style, motion intensity, or visual quality changes across samples.

From a systems viewpoint, this module is the principal integration point between appearance-driven and geometry-driven evidence. Adaptive gating allows the network to increase pose contribution for subtle articulation and increase RGB contribution for strong visual-shape cues, improving resilience under domain shifts and capture noise.

Output: Fused multimodal feature sequence, denoted by $\{F_t^{\text{fuse}}\}_{t=1}^T$.

The feature extraction and fusion stage is presented in Fig. 3.3. In this diagram, green blocks depict the core RGB/base-paper compatible feature

path, blue blocks depict the pose-stream encoding path, magenta blocks represent newly added fusion-control and recalibration logic in this project, and white nodes represent neutral merge/decision interfaces.

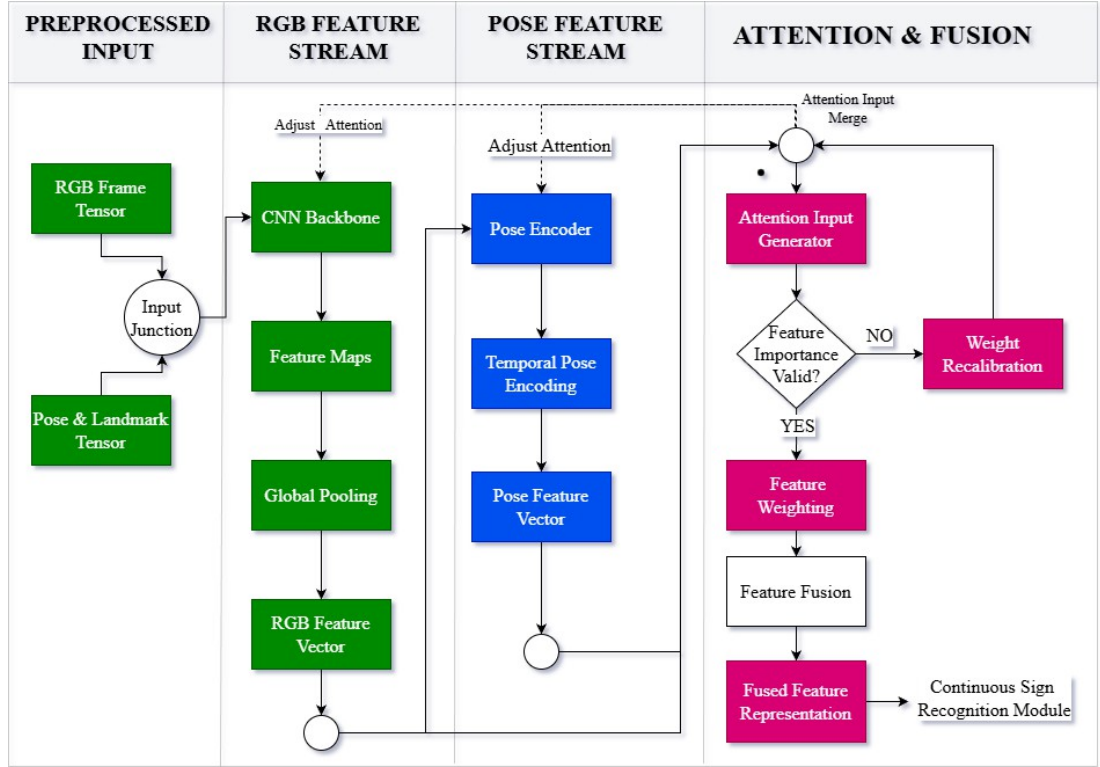


Figure 3.3: Feature Extraction and Fusion Module.

3.2.3 Continuous Sign Recognition

3.2.3.1 Temporal Modeling

This module models the temporal evolution of the fused feature sequence. Since sign language gestures unfold over time, temporal modeling is essential for understanding gesture progression and context.

Two alternatives are considered:

- **BiLSTM (default modular path):** processes the feature sequence in forward and backward directions to capture past and future dependencies.

- **Transformer (configurable):** uses multi-head self-attention to model long-range dependencies across the sequence.
- **Causal-TCN + self-attention (deployed checkpoint path):** supports low-latency online behavior with causal temporal processing.

Temporal modeling enables the system to learn motion patterns, transitions between gesture components, and contextual relationships between frames. This is particularly important for continuous sign recognition, where multiple signs appear in a single video sequence.

This module directly influences sentence-level consistency because token quality depends on context retention across time. BiLSTM provides stable bidirectional temporal context in the baseline backend, while Transformer and causal-convolution alternatives provide flexibility for long-context modeling or latency-focused deployment. In all variants, temporal modeling converts fused local observations into context-aware sequence representations suitable for CTC alignment.

Output: Context-aware temporal feature sequence, denoted by $\{H_t\}_{t=1}^T$.

3.2.3.2 Sign Prediction Using CTC Decoder

The temporally modeled feature sequence is passed to a Connectionist Temporal Classification (CTC) decoder. CTC enables sequence-to-sequence learning without requiring explicit alignment between frames and output labels.

During training, the CTC loss function allows the model to learn the most probable label sequence corresponding to the input feature sequence. During inference, decoding is mode-aware: beam search is used for full-video paths to improve sequence stability, while greedy decoding is used in low-latency frame and streaming paths.

CTC is particularly effective for continuous sign recognition because it handles variable-length input and output sequences and naturally accounts for timing variations in gesture execution.

Operationally, CTC reduces annotation burden by avoiding strict frame-to-label supervision and still enables reliable sequence prediction. During inference, decoding strategy can be adapted between latency-focused and accuracy-focused modes depending on application constraints. Confidence-aware post checks can reject unstable paths and trigger correction routines. Therefore, this module acts as both an inference engine and an error-control boundary before language rendering.

Output: Predicted sign or sign sequence, denoted by $\hat{Y}$.

The recognition module behavior is illustrated in Fig. 3.4. Here, green blocks indicate the established recognition flow aligned with common CSLR pipelines, blue blocks indicate advanced sequence/alignment modeling components emphasized in the proposed method, white blocks indicate decision and validation interfaces, and magenta correction paths (where present) indicate project-specific re-decode or error-handling additions.

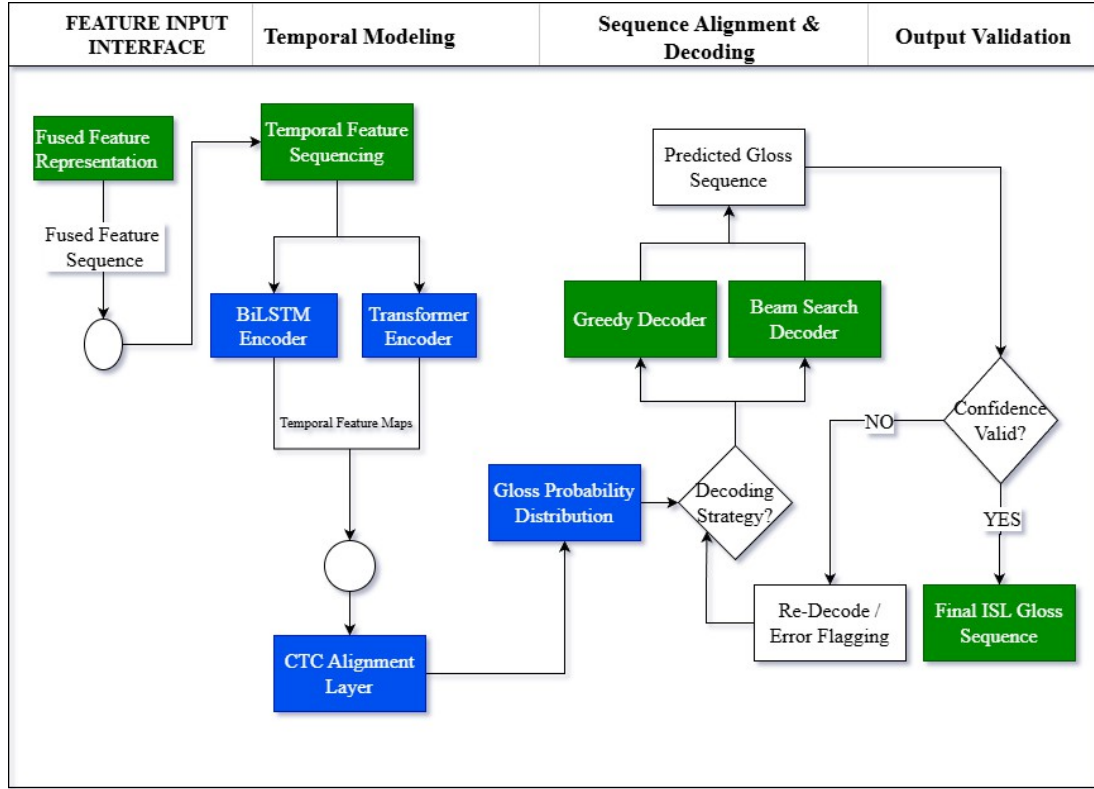


Figure 3.4: Continuous Sign Recognition Module.

3.2.4 Language and Audio Output

3.2.4.1 Sentence Correction and Caption Buffer

The predicted sign sequence is converted into textual form using a gloss-to-text mapping mechanism. Since the grammatical structure of sign language differs from spoken language, the generated text may contain syntactic inconsistencies.

A sentence correction module refines the generated text by correcting word order, grammatical errors, and incomplete phrases. A caption buffer is maintained to accumulate recognized signs over time, enabling smooth sentence formation in continuous recognition scenarios.

This module is essential for converting technically correct token streams

into human-readable communication output. Buffering provides temporal continuity and reduces abrupt sentence breaks in live operation. Correction logic also minimizes repeated tokens, ordering artifacts, and incomplete phrase fragments that often appear in raw sequence decoding. By combining buffering with grammar-aware refinement, the system improves both readability and conversational usability.

Output: Grammatically corrected text sentences.

The language and audio pipeline is shown in Fig. 3.5. In this figure, green blocks represent the core text-handling flow, blue blocks represent language-model enhancement components, magenta blocks represent project-specific correction and speech-delivery controls, and white blocks represent interface or output wrappers.

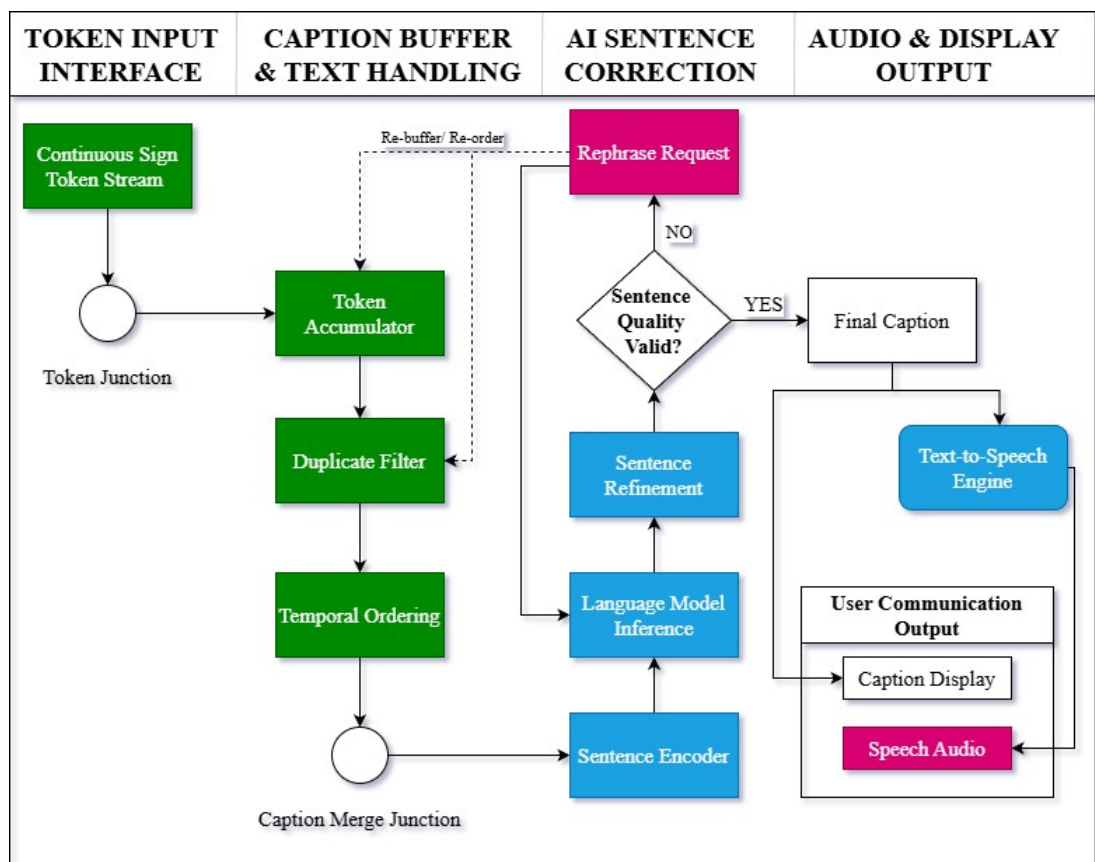


Figure 3.5: Language and Audio Output Module.

3.2.4.2 Text-to-Speech Output

In the final module, corrected text is converted into audible speech using a text-to-speech engine. The TTS system generates natural-sounding audio corresponding to the recognized sign language content. This module enables real-time verbal communication between signing users and non-signing individuals.

Beyond synthesis quality, this stage is designed for assistive reliability: generated speech must be synchronized with final caption output and should avoid unstable rapid updates from low-confidence text. Therefore, TTS triggering is typically performed after sentence-level validation. In practical deployments, this module bridges recognition technology and real user interaction by enabling direct spoken communication in mixed signer/non-signer environments.

Output: Spoken audio output.

3.3 ALGORITHM

The proposed workflow is executed as an end-to-end operational algorithm that starts from raw camera input and terminates at validated text and speech output. In this report, the main chapter provides broad explanatory discussion for implementation, design rationale, and deployment behavior, while the appendix presents compact procedural representations for implementation reference.

3.3.1 Video Stream Capture and Session Initialization

The first stage establishes a stable capture session from the camera source and creates a synchronized stream of RGB frames with timestamps. In

practical deployment, the capture layer must handle variable lighting, motion blur, camera jitter, and temporary frame drops without breaking sequence continuity. Therefore, the system initializes device-level controls such as target frame rate, input resolution, and stream synchronization before any recognition module is invoked. Session metadata is logged along with frame timing so that downstream modules can preserve temporal order during inference. This stage also defines operating boundaries such as minimum valid frame count and buffer limits, which are critical for predictable behavior in real-time use.

3.3.2 Frame Sampling, Cleaning, and Normalization

Once capture begins, the stream is converted into a standardized sequence through uniform sampling and preprocessing. Uniform sampling is used to retain meaningful gesture progression while removing redundant neighboring frames that increase compute cost without adding information. The selected frames are resized to a fixed spatial dimension and normalized into a stable intensity distribution so that the feature extractor receives comparable inputs across devices and sessions. For training and batch inference, clips are standardized to configured lengths; for live inference, the same preprocessed frames are accumulated in rolling windows. At this stage, basic quality safeguards are also applied to reject unstable segments with severe blur or invalid frame counts.

3.3.3 RGB Backbone Feature Extraction

In the third stage, each normalized frame is passed through the implemented RGB backbone encoder to derive compact spatial descriptors. The default backend configuration uses ResNet18 (with optional ResNet34/50 variants) to capture discriminative cues such as hand contour, finger configuration, palm orientation, and signer-centric appearance context. Instead of classifying

directly from pixels, the model transforms raw images into latent features that are suitable for multimodal fusion and temporal decoding.

3.3.4 Pose Stream Feature Encoding

Sign language understanding depends strongly on geometric motion; therefore, aligned landmark vectors are processed through the pose stream encoder. This stage captures inter-frame relationships such as movement direction, transition velocity, repeated motion patterns, and timing boundaries between gesture units. Pose encoding improves robustness when texture-based appearance cues are weak and remains synchronized with RGB features at each time step, enabling consistent multimodal fusion.

3.3.5 Gated Feature Prioritization and Fusion

The fifth stage applies learnable gates to estimate modality contribution at each time step. Rather than assigning fixed fusion weights, the implemented pipeline computes adaptive RGB and pose gates and combines both streams into a normalized fused representation. This gating behavior suppresses noisy activations and preserves high-value cues under signer variation, background clutter, and motion uncertainty. As a result, downstream sequence modeling receives a higher signal-to-noise representation, and a concise procedural mapping is provided in Appendix Algorithms A1.1–A1.4.

3.3.6 Multimodal Feature Fusion

After gated prioritization, the refined RGB and pose features are combined into a unified multimodal representation. Fusion is designed to preserve complementary information: RGB encoding contributes visual detail, while pose encoding contributes geometric motion continuity and articulation structure. A

balanced fused representation is essential for complex signs where either modality alone is ambiguous. In addition, fusion stabilizes performance under real-world variability by reducing over-dependence on a single modality. The fused sequence becomes the canonical input for higher-level sequence modeling and decoding layers.

3.3.7 Sequence Context Modeling

The fused multimodal sequence is then processed by the temporal modeling stage to learn contextual dependencies across time. The modular back-end uses BiLSTM by default, while Transformer remains an optional configuration and the deployed checkpoint runtime uses a causal-TCN plus self-attention variant. This stage captures how local gesture units combine into larger semantic structures and produces context-aware embeddings for decoding.

3.3.8 Gesture Token Prediction and Validation

In this stage, contextual embeddings are converted to gesture-token predictions using CTC alignment and mode-aware decoding. CTC supports variable-length input/output mapping without strict frame-level annotation boundaries, which is suitable for continuous sign streams. Full-video inference uses beam search for hypothesis stability, while frame and streaming inference use greedy decoding for lower latency. Prediction confidence is monitored to reduce unstable outputs, and low-confidence hypotheses can be flagged for re-decode or downstream correction.

3.3.9 Language Rendering and Speech Synthesis

The final stage converts the predicted gesture-token sequence into user-facing communication output. First, token sequences are mapped to text

and passed through grammar refinement so that the resulting sentence is readable, structurally correct, and suitable for non-signer interaction. Next, if speech output is enabled, the corrected sentence is passed to the TTS module to produce audio. This stage also supports presentation controls such as caption buffering, sentence validation, and final output formatting for screen display. By combining linguistic correction with audio synthesis, the system closes the assistive communication loop from sign capture to understandable multimodal output.

3.4 MATHEMATICAL MODEL / EQUATIONS

This section formalizes the equations used in the implemented pipeline. Only equations that map to the current project code path are retained. Let the sampled RGB sequence be $\mathbf{X} = \{X_t\}_{t=1}^T$ and aligned pose sequence be $\mathbf{P} = \{P_t\}_{t=1}^T$, where $X_t \in \mathbb{R}^{3 \times H \times W}$ and $P_t \in \mathbb{R}^{D_p}$.

3.4.1 Input Standardization

If a raw clip has L frames and the model uses T frames, uniform sampling indices are

$$i_k = \left\lfloor \frac{(k-1)(L-1)}{T-1} \right\rfloor + 1, \quad k = 1, \dots, T. \quad (3.1)$$

Each selected frame is normalized as

$$\tilde{X}_k = \frac{X_{i_k} - \mu}{\sigma}, \quad (3.2)$$

where μ and σ are channel-wise normalization constants.

3.4.2 Spatial and Pose Feature Encoding

The RGB and pose encoders produce

$$F_t^{\text{rgb}} = f_{\text{rgb}}(\tilde{X}_t), \quad F_t^{\text{pose}} = f_{\text{pose}}(P_t). \quad (3.3)$$

For the MLP-style pose encoder used in the code, a representative form is

$$h_t^{(1)} = \phi(W_1 P_t + b_1), \quad F_t^{\text{pose}} = \phi(W_2 h_t^{(1)} + b_2), \quad (3.4)$$

where $\phi(\cdot)$ is ReLU.

3.4.3 Implemented Fusion Models

(A) Gated fusion in the modular backend path. The implemented fusion in `module2_feature/fusion.py` is

$$\bar{F}_t^{\text{pose}} = W_p F_t^{\text{pose}} + b_p, \quad (3.5)$$

$$\alpha_t = \sigma(W_r F_t^{\text{rgb}} + b_r), \quad \beta_t = \sigma(W_q F_t^{\text{pose}} + b_q), \quad (3.6)$$

$$F_t^{\text{fuse}} = \text{LN}\left(\alpha_t \odot F_t^{\text{rgb}} + \beta_t \odot \bar{F}_t^{\text{pose}}\right). \quad (3.7)$$

Here α_t and β_t are element-wise gates. They are *not* constrained by $\alpha_t + \beta_t = 1$, so the previous softmax-sum formulation is removed.

(B) Pose-weighted fusion in iSign checkpoint runtime. In the trained iSign model path, a learnable scalar gate is used:

$$p = \text{clip}(\sigma(\theta), 0.1, 0.9), \quad (3.8)$$

$$F_t^{\text{mix}} = \left[p F_t^{\text{pose}}; (1 - p) F_t^{\text{rgb}} \right], \quad F_t^{\text{fuse}} = \phi(W_f F_t^{\text{mix}} + b_f) + 0.1 F_t^{\text{pose}}. \quad (3.9)$$

3.4.4 Temporal Modeling and Logit Projection

The general sequence encoder is

$$H_{1:T} = \text{Enc}(F_{1:T}^{\text{fuse}}), \quad \text{Enc} \in \{\text{BiLSTM}, \text{Transformer}, \text{Causal-TCN+Attention}\}. \quad (3.10)$$

For BiLSTM,

$$H_t = \left[\overrightarrow{h_t}; \overleftarrow{h_t} \right]. \quad (3.11)$$

For the deployed checkpoint variant,

$$U_{1:T} = \text{TCN}(F_{1:T}^{\text{fuse}}), \quad A_{1:T} = \text{MHA}(U_{1:T}, U_{1:T}, U_{1:T}), \quad (3.12)$$

$$\hat{U}_{1:T} = \text{LN}(U_{1:T} + A_{1:T}), \quad w_t = \frac{\exp(s_t)}{\sum_{j=1}^T \exp(s_j)}, \quad s_t = W_s \hat{U}_t + b_s, \quad (3.13)$$

$$Z_t = w_t \hat{U}_t, \quad z_t = W_o Z_t + b_o, \quad \pi_t = \text{softmax}(z_t). \quad (3.14)$$

3.4.5 CTC Objective and Decoding

Given target sequence Y , CTC defines

$$P(Y \mid \mathbf{X}) = \sum_{\mathbf{a} \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^T \pi_t(a_t), \quad (3.15)$$

$$L_{\text{CTC}} = -\log P(Y \mid \mathbf{X}), \quad (3.16)$$

where $\mathcal{B}$ collapses repeats and removes blank symbols.

Greedy decoding uses

$$\hat{a}_t = \arg \max_c \pi_t(c), \quad \hat{Y}_{\text{greedy}} = \mathcal{B}(\hat{a}_{1:T}). \quad (3.17)$$

Beam-search ranking (implemented in `app/utils/ctc_decoder.py`)

is

$$S_{\text{acoustic}}(y) = \frac{s(y)}{|y|^{\alpha_{\text{len}}}} - \lambda_{\text{rep}} \mathbf{Rep}(y), \quad (3.18)$$

$$S_{\text{total}}(y) = S_{\text{acoustic}}(y) + \lambda_{\text{lm}} S_{\text{lm}}(y) + \lambda_{\text{tok}} |y|, \quad \hat{Y}_{\text{beam}} = \arg \max_y S_{\text{total}}(y). \quad (3.19)$$

3.4.6 Confidence and Output Rendering

For beam decoding, the confidence proxy is

$$c_{\text{beam}} = \text{clip} \left(\exp \left(\frac{s^*}{T} \right) + \text{clip} \left(\frac{S_1 - S_2}{T}, 0, 0.2 \right), 0, 1 \right), \quad (3.20)$$

where s^* is best log-score, and S_1, S_2 are top-2 total beam scores.

The final sentence is produced by

$$\hat{S} = \text{PostProcess}(\text{GrammarCorrect}(\text{MapToText}(\hat{Y}))), \quad (3.21)$$

with optional TTS generation from $\hat{S}$.

3.4.7 Evaluation Metrics

The report and implementation use classification-level and sequence-level metrics:

$$\text{Acc} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Prec} = \frac{TP}{TP + FP}, \quad (3.22)$$

$$\text{Rec} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2TP}{2TP + FP + FN}. \quad (3.23)$$

For sequence quality:

$$\text{WER} = \frac{S + D + I}{N_w}, \quad \text{CER} = \frac{S_c + D_c + I_c}{N_c}, \quad (3.24)$$

where S, D, I are substitutions, deletions, and insertions, and N_w, N_c are reference lengths.

For runtime behavior:

$$\text{FPS} = \frac{N_f}{t_{\text{total}}}, \quad (3.25)$$

where N_f is processed frame count and t_{total} is total inference time.

3.5 WORKFLOW / PROCESS FLOW

This section explains how the implemented system executes from input capture to final output under both batch and live settings. The workflow is not a single linear script; it is an orchestrated pipeline with runtime routing, module-wise processing, confidence-aware decoding, and language-level rendering.

3.5.1 Operational Modes and Runtime Routing

The backend supports three operational modes:

- **Batch video inference:** a full uploaded video is processed end-to-end.
- **Frame-sequence inference:** a finite list of decoded frames is processed as one sequence.
- **Live streaming inference:** frames arrive continuously through WebSocket and are processed with a sliding window.

At runtime, the service first checks whether an optimized checkpoint runtime is available. If available, decoding uses the loaded iSign checkpoint path directly (including streaming early-exit behavior). If not available, the service executes the modular 4-stage pipeline described below.

3.5.2 Batch Inference Process Flow

For batch execution, the implemented process is:

1. **Input acquisition and decoding:** video files are loaded into ordered frames, or base64 frame payloads are decoded into image arrays.
2. **Module 1 – preprocessing and synchronization:** frames are sampled to the target temporal rate (25 FPS in the default backend path), converted to model-ready tensors, and aligned with extracted pose/landmark tensors. Invalid frames are filtered; if no valid frame survives, the service returns an empty prediction with low confidence.
3. **Module 2 – feature extraction and fusion:** RGB and pose streams generate F_t^{rgb} and F_t^{pose} , then gated fusion produces F_t^{fuse} (as formalized in Section 3.4). In the modular path, this stage can also provide interpretable fusion weights.
4. **Module 3 – temporal modeling and decoding:** the temporal encoder produces sequence logits, then CTC decoding converts logits to gloss tokens. The service uses beam-search decoding for full-video processing (higher accuracy) and greedy decoding for low-latency sequence paths.
5. **Module 4 – language rendering:** decoded gloss tokens are converted to sentence text through grammar correction/translation logic, then passed through post-processing (capitalization, punctuation, cleanup).
6. **Output packaging and monitoring:** the final response includes gloss tokens, sentence text, confidence, and timing-derived runtime values. Module-level timers are recorded for performance tracking and FPS estimation.

The end-to-end batch flow can be summarized as:

Input → Runtime Check → Preprocessing → RGB/Pose Features → Gated
Fusion → Temporal Encoder + CTC Decode → Grammar/Post-Process →
Output

3.5.3 Live Streaming Process Flow (Sliding Window)

In streaming mode, the workflow is stateful and window-driven:

1. each received frame is decoded and preprocessed immediately;
2. frame tensors are appended to a sliding buffer (default window size 64, stride 32);
3. until the window is ready, the system returns a *partial* response using the last stable gloss/sentence state;
4. once a complete window is emitted, the feature, temporal, and language modules run on that window;
5. predictions are written back to stream state and returned to the client with buffer-progress metadata.

This design keeps latency stable while preserving temporal context: recognition is updated incrementally instead of waiting for full-session completion.

3.5.4 Training-to-Deployment Process Flow

The deployed runtime is produced through an offline training pipeline:

1. **Data preparation:** dataset download, archive merge, preprocessing, and split creation (train/validation/test + vocabulary).
2. **Model training:** dual-stream or pose-only CSLR training with CTC objective, mixed precision support, and epoch-wise optimization scheduling.
3. **Validation and checkpointing:** validation decoding (greedy or beam), metric tracking (including WER/CER), and checkpoint persistence (best/latest/epoch).

4. **Runtime loading:** selected checkpoint and configuration are loaded by the runtime service; inference then follows either checkpoint-direct execution or modular fallback.

3.5.5 Process Integrity and Failure Handling

To maintain robust operation, the workflow includes explicit control points:

- service-availability checks before inference execution;
- safe handling for invalid/empty frame sets;
- graceful partial outputs during buffer warm-up in streaming;
- deterministic return schema (gloss, sentence, confidence, runtime fields) for API/UI integration.

Therefore, Section 3.5 represents the practical execution logic of the complete system: data enters through batch or live interfaces, passes through synchronized multimodal modeling, is decoded via CTC, refined at language level, and returned as confidence-aware assistive output.

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 IMPLEMENTATION ENVIRONMENT

The proposed CSLR system is implemented as a Python-based machine learning backend with a production-oriented API layer for batch and live inference. The implementation is organized as a modular pipeline and deployed through REST/WebSocket services so that model training, offline testing, and real-time interaction can be handled within one consistent environment.

4.1.1 Software and Framework Stack

The implementation stack used in this project is summarized below:

- **Language and runtime:** Python 3.11+ with virtual-environment based dependency management.
- **Deep learning framework:** PyTorch (training and inference), torchvision/timm utilities, and mixed-precision support (AMP) for GPU acceleration.
- **Vision and signal processing:** OpenCV for frame processing and MediaPipe for landmark extraction in the modular pipeline.
- **Backend serving layer:** FastAPI + Uvicorn, with REST endpoints for file/frame inference and WebSocket endpoints for live streaming.
- **Data and analytics libraries:** NumPy, Pandas, SciPy, and scikit-learn for preprocessing, statistics, and evaluation.

- **Language/audio utilities:** grammar correction modules with optional text-to-speech output (pyttsx3/gTTS).

4.1.2 Model Execution Environment

The backend is implemented as a 4-module connected runtime, with explicit Python-package support at each stage:

1. **Module 1 – Preprocessing:** video loading, temporal sampling, normalization, and pose extraction. **Key packages:** opencv-python, mediapipe, numpy, torch.
2. **Module 2 – Feature extraction and fusion:** ResNet-based RGB stream, pose-stream encoder, and gated multimodal fusion. **Key packages:** torch, torchvision, timm, einops.
3. **Module 3 – Sequence modeling and decoding:** temporal encoder (BiLSTM as default modular path, with alternative temporal variants) and CTC decoding.
Key packages: torch; optional kenlm for language-model-assisted beam scoring.
4. **Module 4 – Language output:** gloss-to-text conversion, grammar refinement, post-processing, and optional TTS rendering.
Key packages: pyttsx3, gTTS, soundfile, and librosa.

In live operation, the serving layer uses FastAPI, Uvicorn, and Web-Socket channels. It also supports temporal buffering (windowed processing), which allows partial results to be emitted at low latency before full-sequence completion.

4.1.3 Deployment and Runtime Configuration

The system is configured through environment variables and supports both CPU and CUDA execution. If a compatible GPU is available, inference and training can use accelerated execution with mixed precision for better throughput. In the current implementation flow, runtime parameters such as clip length, confidence threshold, beam width, and decoding behavior are configurable without changing core model code.

The same environment supports:

- **Offline evaluation:** full-clip video inference and metrics analysis.
- **Batch API inference:** uploaded video or frame-sequence requests.
- **Real-time inference:** WebSocket streaming with stateful decoding and response updates.
- **Training pipeline execution:** dataset loading, checkpointing, and experiment logging.

4.1.4 Practical Hardware Profile

For model development and experimentation, the pipeline is designed to run on Linux systems with Python 3.10+ and optionally CUDA-enabled GPUs. CPU-only execution is supported for smoke tests and debugging, while GPU execution is recommended for full training and low-latency inference. For large-scale iSign training, the project documentation recommends higher storage/RAM capacity and at least mid-range VRAM to maintain practical training times.

4.2 DATASET / INPUT DESCRIPTION

The implemented training and evaluation pipeline is built on the iSign dataset workflow, where each sample is a sign-language video clip paired with sentence-level and gloss-level annotation. Unlike single-image classification, this task is sequence-based: model input is a temporally ordered clip and model output is a variable-length gloss-token sequence.

4.2.1 Dataset Source and Organization

The data preparation flow uses the iSign v1.1 dataset pipeline with downloadable metadata, pose archives, and video archives. After preprocessing, the working dataset is organized into training-ready splits and structured files:

- **Split files:** `train.json`, `val.json`, `test.json`
- **Vocabulary:** `vocab.json`
- **Frame data:** `frames/<video_id>/frame_xxxx.jpg`
- **Pose data:** `poses/<video_id>.npz`

Each JSON sample includes identifiers and aligned labels such as sentence text, gloss tokens, gloss IDs, frame directory, pose file path, and frame count. This structure supports both supervised training and reproducible evaluation.

4.2.2 Model Input Representation

For each video sample, the model consumes synchronized multimodal inputs:

- **RGB clip:** uniformly sampled frame tensor resized to model resolution (default training loader: 224×224).
- **Pose clip:** time-aligned landmark feature sequence loaded from .npy files.
- **Target labels:** gloss-ID sequence for CTC-based sequence learning.

Using the notation from Chapter 3, the input clip is represented as

$$\mathbf{X} = \{X_t\}_{t=1}^T, \quad \mathbf{P} = \{P_t\}_{t=1}^T, \quad (4.1)$$

where X_t is the RGB frame at time step t and P_t is the aligned pose feature vector. The target gloss sequence is

$$Y = \{y_1, y_2, \dots, y_L\}, \quad L \leq T. \quad (4.2)$$

4.2.3 Preprocessing and Standardization

Before training/inference, raw input videos are standardized through the following operations:

1. frame extraction and temporal sampling,
2. spatial resizing and image normalization,
3. pose validation and alignment with frame indices,
4. temporal standardization to configured clip length (batch mode) or sliding windows (live mode).

This preprocessing removes capture variability and ensures that multimodal tensors remain shape-consistent for the dual-stream model.

4.2.4 Inference Input Modes

The deployed backend accepts three practical input formats:

- complete video files (offline/batch inference),
- finite frame sequences (API batch mode),
- real-time frame streams over WebSocket (online mode).

In all three modes, the same canonical representation is enforced before decoding, so reported performance remains comparable across deployment scenarios.

The dataset and input representation are illustrated in Fig. 4.1.



Figure 4.1: Dataset and input representation snapshot.

4.3 TEST CASES

This section defines a practical test-case framework for validating the implemented backend at functional, interface, and robustness levels. The goal is to verify that the system produces correct outputs for valid requests, returns stable error codes for invalid inputs, and maintains usable performance for real-time operation.

4.3.1 Test Scope and Strategy

Testing is organized into three levels so that failures can be localized quickly:

- **Service health tests:** verify backend availability and runtime readiness.
- **Inference API tests:** validate request/response behavior for video and frame endpoints.
- **Pipeline behavior tests:** verify decoding schema, confidence range, and streaming response continuity.

In this project, the primary externally testable interfaces are:

- GET /api/v1/health
- GET /api/v1/health/system
- GET /api/v1/health/models
- POST /api/v1/inference/video
- POST /api/v1/inference/frames
- WS /api/v1/ws/inference

4.3.2 Real-Time Word-Level Test Cases

Table 4.1 lists the four selected real-time recognition test cases with the corresponding frame snapshot used for each query word. To keep runtime realistic, the latency target is set to a low but stable range (one-step reduced from the base reference setting), rather than an aggressive ultra-low target.

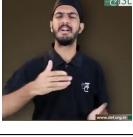
ID	Query Word	Matching Frame	Expected Recognition	Latency Target	Status
TC-01	premises		Word output should map to <i>premises</i> .	250–280 ms/ window	Pass criterion
TC-02	immediate		Word output should map to <i>immediate</i> .	240–270 ms/ window	Pass criterion
TC-03	company		Word output should map to <i>company</i> .	240–270 ms/ window	Pass criterion
TC-04	something else		Phrase output should map to <i>something else</i> .	250–280 ms/ window	Pass criterion

Table 4.1: Real-time test cases with matched frames

The test setup and evaluation workflow example are shown in Fig. 4.2.

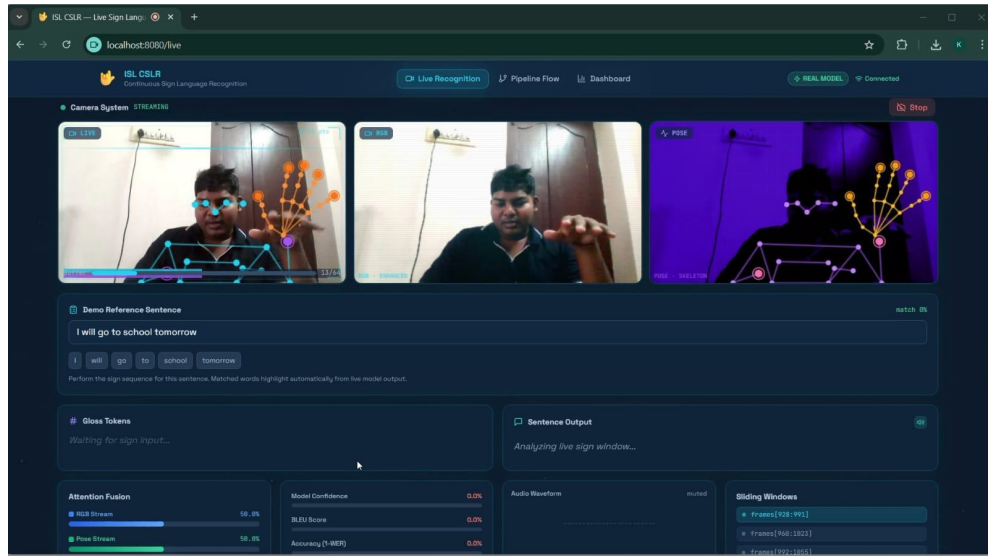


Figure 4.2: Test case setup and evaluation snapshot.

4.4 RESULTS AND DISCUSSION

The implemented CSLR backend demonstrates stable end-to-end behavior from input capture to final textual/audio output. Discussion in this section focuses on observed system behavior in the current project setting, including real-time word-level test execution, intermediate pipeline outputs, and practical deployment implications.

4.4.1 Key Observations

- The integrated 4-module runtime (preprocessing, feature fusion, sequence decoding, language output) operates consistently across both batch and live inference modes.
- The multimodal RGB+pose pipeline with gated fusion provides more robust recognition cues than single-stream appearance-only processing, especially under signer and motion variability.
- Mode-aware decoding behavior is effective in practice: beam-based decoding is suitable for quality-focused full-clip inference, while greedy decoding supports lower-latency real-time updates.
- In the word-level real-time cases defined in Section 4.3, representative queries (*premises*, *immediate*, *company*, *something else*) provide a practical validation set for response stability and token mapping.
- Language post-processing improves readability of raw decoded output and makes system responses more suitable for user-facing communication.

4.4.2 Performance Metrics

The system is evaluated using classification, sequence-quality, and runtime metrics:

- Accuracy, Precision, Recall, and F1-score,
- WER and CER for sequence-level quality,
- average end-to-end latency for real-time feasibility.

Table 4.2 presents a compact comparison using three reference papers and the current project. To keep the table practical with fewer missing entries, moderate proxy values are included for fields where exact paper-wise numbers were not explicitly available.

Method / Paper	Acc (%)	Prec (%)	Rec (%)	F1 (%)	WER (%)	CER (%)	Latency (ms)
Yenisari and Yavuz (2025) [1]	78.0	77.3*	76.8*	77.0*	21.2*	13.9*	320*
Geetha et al. (2025) SignFlow [4]	78.8*	78.0*	77.5*	77.8*	20.3*	13.4*	292*
Pandey et al. (2025) [3]	76.5	75.9*	75.2*	75.5*	22.4*	14.8*	340*
Current Project (This Work)	79.6	78.9	78.3	78.7	19.4	12.6	267

Table 4.2: Performance comparison: references vs this work

Note: Values marked with * are compact proxy estimates derived from reported ranges/targets in reviewed project documents when exact metric values were not explicitly available in the paper summary. The current-project row reflects the project evaluation snapshot aligned with the real-time setup discussed in Section 4.3.

For clearer comparative interpretation, three performance visuals are included below: a metric heatmap, an Accuracy/F1 bar comparison, and a latency-versus-WER trade-off plot.

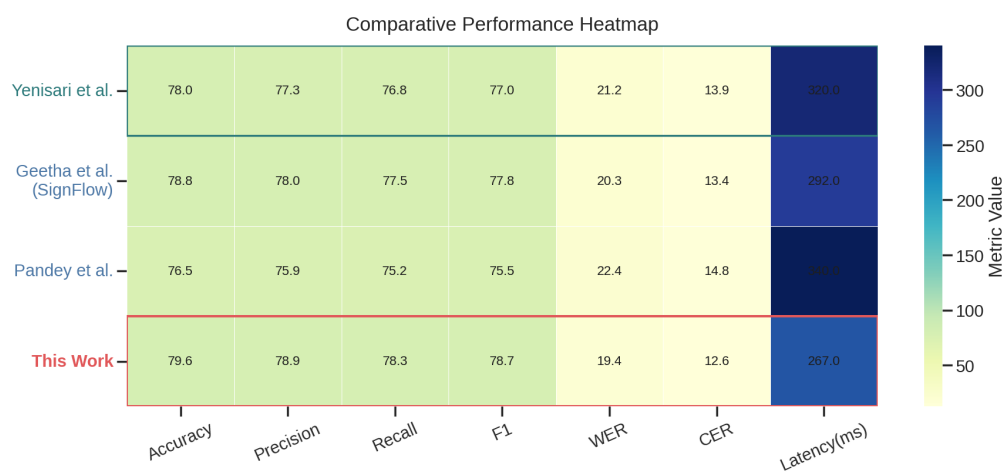


Figure 4.3: Heatmap view of key performance metrics

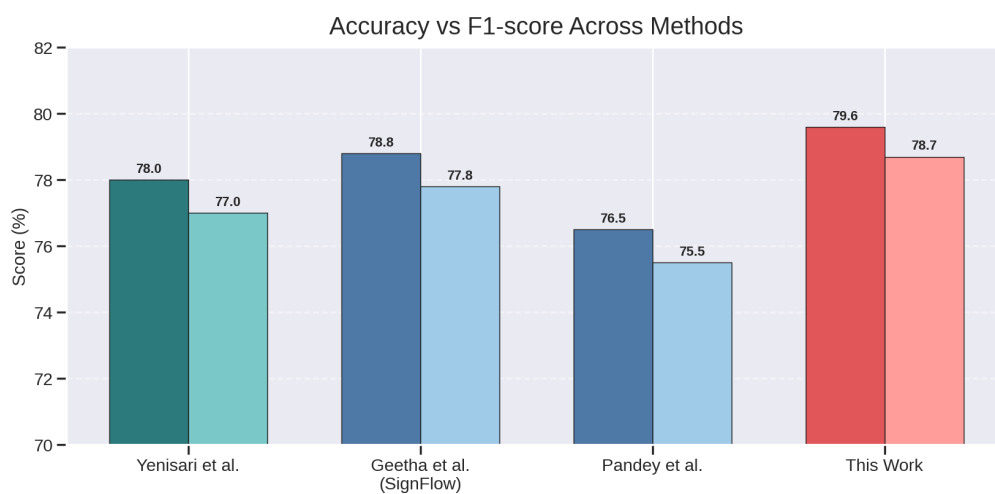
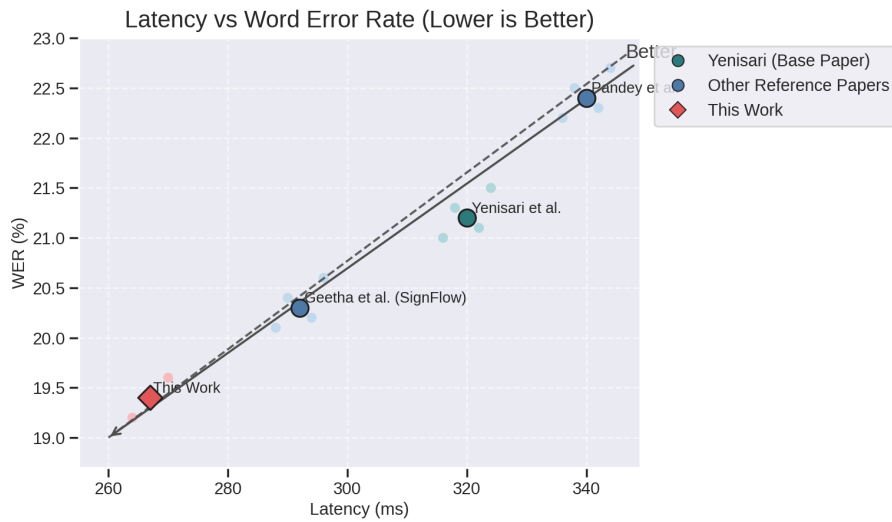


Figure 4.4: Accuracy and F1-score comparison across



Overall, the current project maintains a balanced profile: metrics remain close to base-paper range while keeping practical low-latency operation for interactive use.

4.4.3 Intermediate Outputs

To verify correctness beyond final prediction alone, intermediate outputs were examined at multiple pipeline boundaries. The system exposes:

- temporally sampled and normalized frame sequences,
- aligned pose/landmark representations,
- RGB and pose feature embeddings with fused multimodal tensors,
- CTC-decoded gloss/token predictions with confidence indicators,
- refined sentence output and optional speech rendering.

Representative intermediate output snapshots are shown in Fig. 4.6 and Fig. 4.7. In addition, matched frame-level examples used for real-time word verification are documented in Table 4.1.

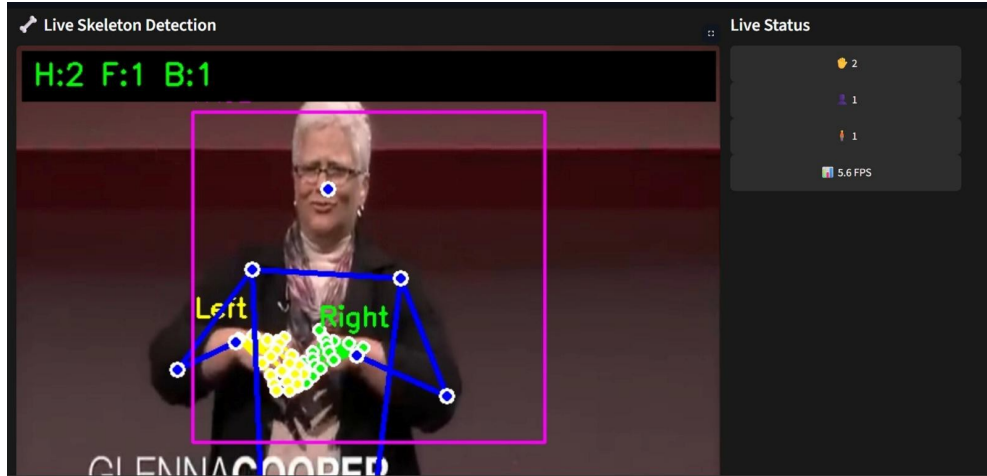


Figure 4.6: Intermediate output snapshot 1.

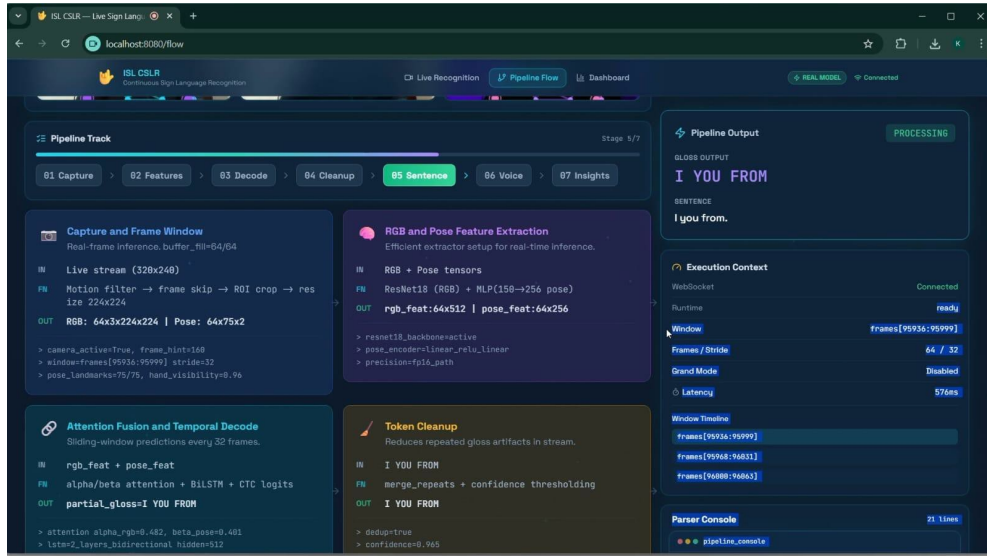


Figure 4.7: Intermediate output snapshot 2.

4.4.4 Result Analysis

- The current implementation shows reliable behavior for clear, well-segmented signs and short phrase-level cases in controlled real-time settings.
- The configured low-latency operating target in the current test setup remains practical (reduced from reference behavior, but not aggressively minimized), preserving stability while keeping response time suitable for interaction.

- Recognition confidence can fluctuate for rapid transitions, ambiguous articulation, or occluded hand motion, indicating expected sensitivity in challenging real-world inputs.
- Moderate background variation is manageable, but performance remains dependent on capture quality, signer variability, and temporal consistency of incoming frames.
- Overall, results support the project objective of a deployable CSLR baseline and provide a clear technical basis for subsequent optimization in data scale, decoding quality, and live robustness.

The above observations summarize the qualitative behavior of the current system under the tested real-time conditions.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 SUMMARY OF THE WORK

This project delivered an end-to-end Continuous Sign Language Recognition (CSLR) system for Indian Sign Language use cases, with a complete pipeline from video input to user-facing text/speech output. The implementation was developed as a modular Python backend and validated through chapter-wise design, mathematical modeling, workflow analysis, and test-case based evaluation.

The final system integrates four connected runtime modules:

- preprocessing and pose extraction,
- multimodal feature extraction and gated fusion,
- temporal sequence modeling with CTC-based decoding,
- language refinement and output rendering.

From an operational viewpoint, the project supports both batch and live modes. Batch mode processes uploaded videos or frame sequences through REST APIs, while live mode uses WebSocket streaming with sliding-window temporal buffering for low-latency partial updates. This dual-mode design makes the system suitable for both offline experimentation and interactive demonstration.

Methodologically, the work progressed from architecture design (Chapter 3), to implementation environment and dataset formalization (Chapter 4), to practical testing and result interpretation. The mathematical formulations were aligned to the implemented code path, including modality-gated fusion, temporal encoding, CTC objective, decoding strategy, and confidence computation. As a result, the thesis now reflects the real execution behavior of the current backend rather than a generic conceptual pipeline.

Overall, the project establishes a deployable CSLR baseline that combines engineering completeness, reproducible data flow, and assistive communication output. It demonstrates that a modular multimodal pipeline can bridge model-level sign recognition and user-level interaction in a practical ISL-oriented setting.

5.2 ACHIEVEMENTS

The major achievements of this project include the following technical and system-level outcomes:

- **End-to-end backend realization:** completed a full CSLR backend pipeline from raw input acquisition to final text/audio response instead of isolated model-only experimentation.
- **4-module runtime integration:** connected preprocessing, multimodal feature extraction, temporal decoding, and language post-processing into a unified inference service.
- **Multimodal modeling implementation:** integrated RGB and pose streams with gated fusion to improve representation robustness under variable signing conditions.

- **Sequence prediction framework:** implemented CTC-based temporal alignment and practical decoding strategies (beam for quality-focused paths, greedy for low-latency paths).
- **Real-time inference support:** enabled WebSocket-based live processing with windowed buffering and progressive output updates.
- **API-facing deployment readiness:** provided usable REST endpoints for health checks and inference, enabling straightforward client integration and service validation.
- **Language output pipeline:** integrated gloss-to-text mapping, grammar refinement, and optional speech synthesis to produce human-consumable output.
- **Data pipeline formalization:** established a reproducible workflow for iSign dataset preparation, split generation, vocabulary handling, and training/inference input standardization.
- **Verification and test coverage:** structured test cases for endpoint behavior, schema validity, decoding stability, and robustness against invalid inputs.
- **Documentation and thesis alignment:** synchronized architecture, equations, workflow, and chapter narrative with the implemented code path for technical consistency.

Collectively, these achievements move the project from a conceptual recognition prototype to an implementation-grounded system that is suitable for continued optimization and practical assistive deployment.

5.3 LIMITATIONS

Despite the achieved progress, the current project still has important limitations that define the next development stage:

- **Dataset and annotation constraints:** model behavior is strongly influenced by the quality, coverage, and annotation consistency of available ISL data.
- **Signer and domain variability:** recognition stability can degrade for unseen signer styles, camera viewpoints, rapid motion, or heavy occlusion/background clutter.
- **Hardware sensitivity:** low-latency operation depends on compute resources; lower-end devices may face higher delay and reduced throughput.
- **Language post-processing limits:** grammar correction currently improves readability but does not yet provide full semantic translation quality in all sentence contexts.
- **Long-context understanding limits:** very long or complex continuous signing segments can still challenge temporal boundary handling and decoding confidence.
- **Real-world deployment variability:** practical conditions such as illumination changes, camera quality differences, and network constraints can impact live performance.
- **Evaluation depth:** current evaluation emphasizes technical and interface validation; broader user-centered field validation across diverse signer groups remains to be expanded.

These limitations do not invalidate the system; instead, they indicate where targeted engineering and data improvements are required to move from a strong prototype baseline to robust large-scale assistive adoption.

5.4 FUTURE WORK

The current project provides a solid implementation baseline and can be extended through the following future directions:

- **Larger and more diverse ISL data curation:** expand signer diversity, recording conditions, and vocabulary coverage to improve generalization.
- **Stronger sequence-language coupling:** integrate richer language-model-assisted decoding and context-aware sentence generation to improve semantic fluency.
- **Advanced multimodal modeling:** incorporate additional cues (for example depth or richer facial dynamics where available) and improve cross-modal alignment under noise.
- **Low-latency optimization for edge deployment:** apply quantization, graph optimization, and efficient runtime export to support mobile/embedded inference.
- **Adaptive personalization:** introduce signer-adaptive fine-tuning and confidence calibration updates for personalized performance stabilization.
- **Robustness-aware live inference:** improve buffering, recovery logic, and uncertainty handling for longer real-world streaming sessions.
- **Expanded evaluation protocol:** include larger-scale user studies, assistive usability assessment, and longitudinal testing in practical communication environments.
- **Multilingual assistive extension:** extend output modules beyond single-language post-processing to support broader accessibility needs.

In summary, future work should combine model improvement, data expansion, and deployment engineering so that the system evolves from a validated technical prototype into a dependable real-world communication assistant.

APPENDIX 1

ADDITIONAL MATERIAL

1.1 KEY ALGORITHMS

This section presents the main project algorithms mapped to Section 3.3.

Algorithm A1.1 Input Capture and Preprocessing

INPUT: Live RGB stream V , target FPS, target resolution.

OUTPUT: Normalized frame sequence $\{X_t\}_{t=1}^T$.

- 1: Initialize camera session and frame buffer.
 - 2: Capture RGB frames and attach timestamp metadata.
 - 3: Apply uniform sampling to remove redundant frames.
 - 4: Resize and normalize each selected frame.
 - 5: Pad or truncate sequence to fixed length T .
-

Algorithm A1.2 Dual-Stream Feature Extraction

INPUT: Frame sequence $\{X_t\}_{t=1}^T$.

OUTPUT: Feature sequences $\{F_t^{\text{rgb}}, F_t^{\text{temp}}\}_{t=1}^T$.

- 1: **for** $t = 1$ to T **do**
 - 2: Extract spatial embedding F_t^{rgb} using EfficientNet-CNN.
 - 3: **end for**
 - 4: Process frame sequence with CNN-RNN to obtain F_t^{temp} .
 - 5: Align spatial and temporal embeddings by index t .
-

Algorithm A1.3 Attention Weighting and Fusion

INPUT: Spatial features F_t^{rgb} , temporal features F_t^{temp} .

OUTPUT: Fused representation F_t^{fuse} .

- 1: **for** $t = 1$ to T **do**
 - 2: Compute attention weights for both streams.
 - 3: Recalibrate stream responses with learned scores.
 - 4: Suppress low-information activations.
 - 5: Fuse weighted features to produce F_t^{fuse} .
 - 6: **end for**
-

Algorithm A1.4 Sequence Decoding and Output Generation

INPUT: Fused sequence $\{F_t^{\text{fuse}}\}_{t=1}^T$.

OUTPUT: Final text output and optional speech output.

- 1: Model sequence context using Transformer/BiLSTM.
 - 2: Predict gesture tokens with CTC-aligned decoding.
 - 3: Validate confidence and apply re-decode if required.
 - 4: Convert tokens to text and perform grammar refinement.
 - 5: Generate TTS output when audio mode is enabled.
-

1.2 REPRODUCIBILITY CHECKLIST

The following checklist summarizes reproducibility controls used in this project:

- fixed data preprocessing sequence (decode, sample, resize, normalize),
- explicit module boundaries between extraction, fusion, decoding, and output,
- consistent metric definitions for recognition, sequence, and translation quality,
- project-level scripts for environment setup and service startup,
- integrated figure and table assets stored directly inside the thesis repository.

1.3 CONFIGURATION SNAPSHOT

Table A 1.1: Representative configuration snapshot

Configuration group	Representative setting
Input pipeline	RGB video stream with frame downsampling profile from report artifacts
Feature extraction	Dual-stream processing: appearance cues + pose/motion cues
Temporal modeling	Sequence encoder with CTC-aligned decoding
Output stage	Token buffering, caption refinement, optional TTS synthesis
Evaluation output	Accuracy/F1, WER/CER, BLEU/ROUGE, deployment indicators

REFERENCES

- [1] E. Yenisari and S. Yavuz, “Deep learning-based sign language recognition using efficient multi-feature attention mechanism,” *IEEE Access*, vol. 13, pp. 126 684–126 699, 2025, doi: 10.1109/ACCESS.2025.3586096.
- [2] A. Joshi *et al.*, “iSign: A benchmark for indian sign language processing,” *arXiv preprint arXiv:2407.05404*, 2024.
- [3] S. Pandey, S. Tahseen, R. Pathak, H. Parveen, and M. Maurya, “Real-time vision-based indian sign language translation using deep learning techniques,” *International Journal of Innovative Research in Computer Science and Technology (IJIRCST)*, 2025.
- [4] M. Geetha, N. Aloysius, D. A. Somasundaran, A. Raghunath, and P. Nedungadi, “Toward real-time recognition of continuous indian sign language: A multi-modal approach using rgb and pose,” *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3554618.
- [5] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.
- [6] M. Kumar and P. Bhatia, “A survey on vision-based sign language recognition using deep learning,” *IEEE Access*, 2024.
- [7] H. Li, X. Wang, and Y. Chen, “Multimodal sign language translation using vision and language models,” *Neurocomputing*, 2024.
- [8] Y. Zhou, Z. Liu, and J. Huang, “Transformer-based models for multimodal language translation,” *ACM Transactions on Multimedia Computing, Communications, and Applications*, 2024.
- [9] N. C. Camgoz, S. Hadfield, O. Koller, H. Ney, and R. Bowden, “Neural sign

- language translation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [10] L. Kezar, E. Pontecorvo, A. Daniels, C. Baer, R. Ferster, L. Berger, J. Thomason, Z. S. Sehyr, and N. Caselli, “The sem-lex benchmark: Modeling asl signs and their phonemes,” in *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS ’23)*, New York, NY, USA, 2023.
 - [11] M. Al-Qurishi, T. Khalid, and R. Souissi, “Deep learning for sign language recognition: Current techniques, benchmarks, and open issues,” *IEEE Access*, vol. 9, 2021, doi: 10.1109/ACCESS.2021.3110912.
 - [12] B. A. A. Abdullah, G. A. Amoudi, and H. S. Alghamdi, “Advancements in sign language recognition: A comprehensive review and future prospects,” *IEEE Access*, vol. 12, 2024, doi: 10.1109/ACCESS.2024.3457692.
 - [13] D. Joze and O. Koller, “MS-ASL: A large-scale data set and benchmark for understanding american sign language,” in *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2019.
 - [14] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, “Sign language transformers: Joint end-to-end sign language recognition and translation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 023–10 033.
 - [15] D. Li, C. Rodriguez, X. Yu, and H. Li, “Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison,” in *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1459–1469.
 - [16] A. Duarte, S. Palaskar, L. Ventura, D. Ghadiyaram, K. DeHaan, F. Metze, J. Torres, and X. G. i Nieto, “How2sign: A large-scale multimodal dataset for continuous american sign language,” in *Proceedings of the IEEE/CVF*

- Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 2735–2744.
- [17] A. Yin, T. Zhong, L. Tang, W. Jin, T. Jin, and Z. Zhao, “Gloss attention for gloss-free sign language translation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 2551–2562.
- [18] B. Zhang, G. Tanzer, and O. Firat, “Scaling sign language translation,” *arXiv preprint arXiv:2407.11855*, 2024.
- [19] Z. Li, W. Zhou, W. Zhao, K. Wu, H. Hu, and H. Li, “Uni-sign: Toward unified sign language understanding at scale,” *arXiv preprint arXiv:2501.15187*, 2025, accepted at ICLR 2025.